

P F E

iTeam University

MASTER BIG DATA



Portraits de Reconnaissance Faciale Automatisée : Méthodes Deep Learning pour l'Analyse des Traits

Auteur

Ines Khelifi

Le 17 Juillet 2022

Niveau

Mémoire de fin d'études

Specialité

Big Data

Organisation

Neivos Consulting

Composition du jury :

Hedi RAGAD
iTeam University

Président du jury

Nadia CHERICHI
iTeam University

Examineur

Abdelkarim MARS
iTeam University

Encadrant

Résumé

Un système de reconnaissance faciale est une technologie capable de faire la correspondance entre les visages présent sur une image numérique avec une base de données préalable. Généralement utilisée pour s'assurer de l'identité des personnes dans les processus d'authentification et de sécurité biométriques, ces algorithmes fonctionnent en identifiant et en mesurant les traits des visages à partir d'une image donnée.

Les recherches sur les algorithmes de la reconnaissance faciale ont commencé dans les années 1960 mais ce n'est que récemment qu'on a vu un essor considérable dans le déploiement des systèmes de reconnaissance faciale.

Non seulement les technologies de vision par ordinateur sont plus faciles à mettre en place de nos jours mais elles nous permettent également de mieux discerner des images dans des conditions extrêmes ou qui posent défi.

Étant donné que la reconnaissance faciale informatisée implique la mesure des caractéristiques physiologiques d'un être humain, les systèmes de reconnaissance faciale sont classés comme biométriques. Bien que la précision des systèmes de reconnaissance faciale en tant que technologie biométrique soit inférieure à la reconnaissance de l'iris et aux empreintes digitales, elle est largement adoptée en raison de son mécanisme qui permet d'éviter le contact humain.

Keywords : image detection, recognition, classification, optimisation

Abstract

Deux des aspects importants dans le cadre de recherche sur la reconnaissance faciale par ordinateur sont abordés ici : la détection des visages et des caractéristiques faciales d'une part, et les méthodes de comparaison et d'identification des visages d'autre part. Les meilleurs résultats apportés au problème de reconnaissance faciale sont obtenus avec les méthodes deep learning notamment avec les classifieurs SVM. Dans cette approche, la détection globale des visages, la localisation des caractéristiques faciales et la comparaison des visages sont effectuées en une seule étape. Ce rapport décrit la mise en place d'une application de détection faciale et d'analyse de données. D'une part, nous décrivons une technique d'apprentissage visuel et la manière avec laquelle on peut l'appliquer à la détection de visage dans diverses conditions. D'autre part, un algorithme de tri et d'analyse des données est implémenté afin de présenter des résultats en temps réel et permettre leur analyse statistique. Enfin, nous apportons une extension au projet en connectant la base de données à la plateforme Cloud via les connecteurs GDBC de Confluent ¹ qui permettent d'automatiser l'analyse des données en temps réel (voir article en annexe ²).

Keywords : computer vision, perception, modelisation

1. <https://www.confluent.io/fr-fr/product/confluent-connectors/>

2. Projet Cloud PFA v2.1 : Connection du Framework Superset à Confluent via Docker Bridges

Table des matières

Résumé	i
Abstract	ii
Table des matières	ii
Liste des figures	v
1 Introduction	1
1 Introduction	2
2 Pourquoi la vision par ordinateur représente un tel défi	2
3 Le processus de reconnaissance faciale	3
4 La collecte des visages	3
5 Detection faciale	3
6 Pré-processing	4
7 Algorithmes	4
7.1 Basés sur l'apparence	4
7.2 Apparence Active	5
7.3 Support vector machines	5
7.4 Méthode Bayesienne	6
7.5 Neural Networks	6
7.6 Basés sur la texture	7
7.7 Basés sur les caractéristiques	7
2 Techniques de Reconnaissance Faciale	8
1 Reconnaissance faciale	9
2 Eigen Face	9
3 Sub-espace facial	10
4 Approche basée sur des position à angles multiples	13
5 Traçage du visage à partir d'une video sous différents angles	14
6 Conclusion	14
3 Réalisation	16
1 Introduction	17
2 Reconnaissance faciale avec Keras, Dlib et OpenCV	17
2.1 Etape 1 : Détection	18
2.2 Etape 2 : CNN	18
2.3 Etape 3 : Vecteurs d'intégration	18

3	Architecture CNN et entraînement	18
3.1	Définition du CNN	18
3.2	Description de l'architecture CNN utilisée	19
4	Comprendre les incorporations de reconnaissance faciale d'apprentissage en pro- fondeur	21
5	Alignement du visage	21
6	Embedding Vectors	22
7	Seuil de la distance	24
8	Reconnaissance Faciale	26
9	Visualisation des Données	27
4	Mise en place de l'application de Reconnaissance Faciale	30
1	Planification	31
2	Description de l'architecture de l'application	31
3	Live Camera Recognition	31
3.1	Eigenfaces	32
3.1.1	Exemple	32
3.1.2	Description algorithmique de la méthode EigenFaces	32
3.2	Fisherfaces	34
3.2.1	Exemple	34
3.2.2	Description algorithmique de la méthode Fisherfaces	34
3.3	Local Binary Patterns Histograms	36
3.3.1	Exemple	36
3.3.2	Description algorithmique de la méthode LBPH	38
4	CNN Face Recognition	40
4.1	Description du mode de fonctionnement de keras	40
4.1.1	Loss Fonction	42
4.1.2	Optimizer	43
5	Analyse des données	43
6	Visualisation des données avec PowerBI	43
6.1	Représentation par spécialité	44
6.2	Représentation par matière fréquentée	45
7	En Résumé	45
	Conclusion et perspectives	47
1	Conclusion d'étude	47
2	Ouverture et perspectives	48
	Documents en Annexes	49
	Bibliographie	50

Liste des figures

1.1	Zero-light 3D face biometric technology Biometric update	3
1.2	Exemple de pré-traitement d'image (a) Initial Image. (b) Pixelated Image.	5
1.3	SVM Classifier	6
1.4	Deep Convolutional Neural Networks for Quantum Computers by Jonas Landman Towards Data Science	7
2.1	A two pixel image mapped to a plane.	9
2.2	Sub-espace Facial ou manifold (a) Face vs non face Manifolds. (b) Face Manifolds ou individus différents	10
2.3	Non linéarité et non convexité des manifolds faciales sous translation (du haut vers le bas), rotation, mise à l'échelle et transformations gamma	12
2.4	Parametric versus view-based eigenspace methods. a Reconstructions of the input image (left) with parametric (middle) and view-based (right) eigenspaces. Top : training image; bottom : novel (test) image. b Difference in the way the two approaches span the manifold	13
2.5	Feature extraction. We first obtain the texture map of the human head on the surface of a spherical model through back-projection of multi-view images captured by the camera network, then represent it with spherical harmonics.	14
3.1	Principes de la reconnaissance faciale	17
3.2	Comparaison des convolutions LeNet et AlexNet	19
3.3	Alignement du visages dans les photos	22
3.4	Embedding vectors	23
3.5	Distance Threshold	24
3.6	Distance distributions of positive and negative pairs	25
3.7	Exemple de reconnaissance faciale par SVM	27
3.8	Exemple de représentation t-SNE pour l'analyse des cellules à différents stades de développements	28
3.9	t-distributed Stochastic Neighbor Embedding	29
4.1	Schéma résumant la structure de l'application mise en place pour la reconnaissance faciale	31
4.2	Représentation des taux de reconnaissance pour les méthodes eigenface et fisherface	37
4.3	Opérateur LBP avec un voisinage 3x3	38
4.4	Alignement de pixels sur un cercle	39
4.5	Enhanced Local Binary histogram	40
4.6	Représentation graphique du réseau de neurone séquentiel	41
4.7	Fonction d'activation du réseau neuronal	42

4.8	Un exemple de neurone montrant l'entrée $(x_1 - x_n)$ et les poids correspondants $(w_1 - w_n)$, un biais (b) et la fonction d'activation f à la somme amortie des données à l'entrée.	42
4.9	Tableau de bord montrant le nombre de passage des étudiants par spécialités (les données utilisées ici sont des données théoriques juste à titre d'exemple)	44
4.10	Représentation statistique de la fréquentation journalière et par spécialité.	45

Chapitre 1

Introduction

Objectifs

Dans ce chapitre nous allons définir les différents algorithmes de reconnaissance faciale qui peuvent être utilisés pour identifier des photos ou des vidéos en temps réel. Les avantages et les inconvénients de cette technologie et quels sont les défis rencontrés lors de l'implémentation de certains de ces algorithmes.

Sommaire

1	Introduction	2
2	Pourquoi la vision par ordinateur représente un tel défi	2
3	Le processus de reconnaissance faciale	3
4	La collecte des visages	3
5	Détection faciale	3
6	Pré-processing	4
7	Algorithmes	4
7.1	Basés sur l'apparence	4
7.2	Apparence Active	5
7.3	Support vector machines	5
7.4	Méthode Bayésienne	6
7.5	Neural Networks	6
7.6	Basés sur la texture	7
7.7	Basés sur les caractéristiques	7

1 Introduction

Initialement définie comme une branche de l'intelligence artificielle pour permettre aux robots de percevoir visuellement, la reconnaissance faciale fait maintenant partie d'une discipline plus généralisée et plus large de la vision par ordinateur. Les applications de la vision par ordinateur peuvent traiter des images s'étalant sur une large gamme du spectre électromagnétique. Les rayons X sont utilisés dans le domaine médical pour créer des images du corps humain sans avoir besoin de recourir à la chirurgie. Les rayons gamma et les ondes radio en imagerie par résonance magnétique (IRM) capturent des images de fines sections du corps humain utiles pour le diagnostic et le traitement des maladies. [Chen, 2013].

D'autre part les Rayons X dans l'industrie automobile sont utilisés pour l'inspection de matériaux difficiles à détecter à première vue, tels les fractures ou les fissures, les vides et autres défauts...

Contrairement à la vision par ordinateur, les applications de reconnaissance faciale sont confinées à la bande étroite du spectre lumineux où l'authentification biométrique a lieu.

La biométrie est un terme utilisé pour décrire certaines caractéristiques humaines telles que l'iris, les empreintes digitales ou la géométrie de la main. Ces métriques sont utilisées pour l'identification et le contrôle d'accès des individus à des bâtiments sous surveillance. Le visage devient la mesure de détection préférée par rapport à la biométrie actuelle simplement parce qu'il s'agit d'un reflet direct de l'identité de la personne et que sa nature non intrusive offre plus de commodité et de facilité de vérification. Par exemple, dans un système d'empreintes digitales, le sujet doit interagir avec le système en plaçant un doigt sous un lecteur d'empreintes digitales, et les résultats doivent être vérifiés par un expert. En revanche, utiliser la détection faciale comme métrique ne nécessite aucune intervention et les résultats peuvent être vérifiés par un non-expert.

2 Pourquoi la vision par ordinateur représente un tel défi

Toutes les images doivent d'abord être capturées par une caméra, puis transmises à une application d'identification et de classification pour un traitement ultérieur. Comparé au système visuel humain, la caméra représente l'œil et le logiciel représente le cerveau de l'application. Pour acquérir l'image, la caméra utilise la lumière réfléchie par un objet et transmet l'intensité lumineuse à ses capteurs intégrés. Les capteurs convertissent ensuite chacune des intensités en une valeur comprise entre 0 et 255, et cette matrice devient la représentation finale de l'image capturée. Notez que la lumière est une forme d'énergie électromagnétique couvrant une gamme de fréquences connue sous le nom de spectre visuel. De plus, les capteurs sont uniques aux appareils de photo numériques, car les anciennes caméras analogiques capturaient les images sur un film.

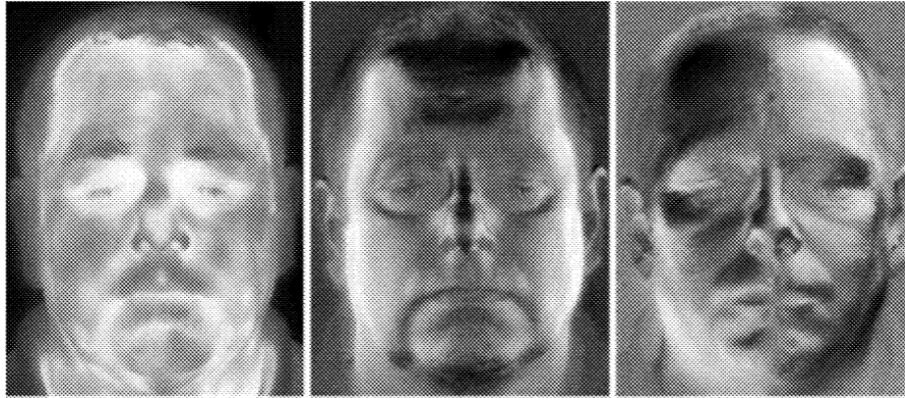


FIGURE 1.1 – Zero-light 3D face biometric technology | Biometric update

3 Le processus de reconnaissance faciale

La reconnaissance faciale consiste à étiqueter un visage comme reconnu ou non reconnu. Le processus a un cycle de vie basé sur la collecte, la détection, le prétraitement et la reconnaissance des images. Lors de l'étape de collecte, les images sont capturées et stockées pour ensuite faire objet de détection et de reconnaissance. Dans la phase de détection, les régions d'un visage dans une image sont identifiées et leur emplacement est enregistré. L'étape de prétraitement modifie l'image en supprimant les caractéristiques indésirables telles que l'ombrage ou l'éclairage excessif. La reconnaissance, qui constitue la dernière étape du cycle, identifie le visage comme reconnu ou non reconnu.

4 La collecte des visages

Avant qu'un système de reconnaissance puisse identifier un visage, il doit d'abord être entraîné sur une collection d'images, appelée ensemble d'entraînement. L'ensemble permet la comparaison du contenu avec une nouvelle image pour déterminer si la différence est suffisamment petite pour une identification positive. Pour une reconnaissance réussie, l'ensemble doit être robuste, c'est-à-dire qu'il doit contenir une variété d'images telles que des images faciales (échantillons positifs) ainsi que des images non faciales (échantillons négatifs) telles que des voitures, des arbres, etc. De plus, l'ensemble doit contenir une variation d'images faciales, où le sujet regarde vers le haut ou vers le bas, avec différentes variations des expressions du visage et des conditions d'éclairage. Il est important d'avoir un ensemble varié et diversifié plutôt qu'un grand nombre d'images avec peu ou pas de variation.

5 Detection faciale

La détection faciale consiste à la localisation d'un visage dans une image sans identification. Bien que de nombreux algorithmes de détection de visage existaient avant 2001, une percée majeure dans la détection faciale est apparue avec l'article de Viola-Jones "Rapid Object Detection

using a Boosted Cascade of Simple Features" [Viola, 2001b ; Lienhart, 2002].

Contrairement aux méthodes de détection faciale précédentes qui reposaient sur l'analyse des pixels, Viola-Jones a conçu un algorithme appelé "Haar-classifier" qui s'appuyait sur des fonctionnalités de type Haar. Le classificateur de Haar est un algorithme d'apprentissage automatique formé avec de nombreux échantillons positifs et négatifs pour détecter des objets dans des images. Pour que le classificateur fonctionne correctement, la taille de l'image dans l'ensemble d'apprentissage doit être la même que la taille de l'image d'entrée utilisée pour la détection des objets.

6 Pré-processing

Les algorithmes de reconnaissance faciale sont sensibles à de nombreux facteurs externes tels que l'orientation de la tête, l'occlusion partielle, l'expression du visage et les conditions d'éclairage. Pour minimiser ces effets sur la performances de l'algorithme et réduire les erreurs, les images faciales sont prétraitées pour les rendre faciles à reconnaître. Une technique de prétraitement standard consiste à réduire la luminosité en utilisant un histogramme égalisé. L'histogramme de l'image est produit par un décompte de valeurs de pixel dans l'intervalle de 0 à 255. Si la plupart des cases hautes sont à droite de l'histogramme, l'image est lumineuse et si la plupart des cases hautes sont à gauche de l'histogramme, l'image est sombre. L'égalisation d'un histogramme répartit les bacs uniformément sur l'image, ce qui lui donne un bon contraste.

7 Algorithmes

Voici une liste d'approches courantes pour la conception d'algorithmes de reconnaissance faciale.

7.1 Basés sur l'apparence

Les algorithmes basés sur l'apparence utilisent les données de pixels d'image dans leur ensemble pour la reconnaissance.

La corrélation directe, Eigen-face et Fisher-Face appartiennent à cette classe d'algorithmes. La corrélation directe utilise la comparaison directe des pixels d'image de deux images faciales, produisant un score de similarité [Brunelli, 1993 ; Nefian, 1996]. Contrairement à une méthode de corrélation directe qui utilise des images faciales dans leur espace d'image d'origine, les algorithmes Eigen-face et Fisher-face réduisent l'image au facteur le plus discriminant et effectuent la comparaison l'images dans un espace de dimension réduite [Turk, 2001 ; Belhumeur, 1997].

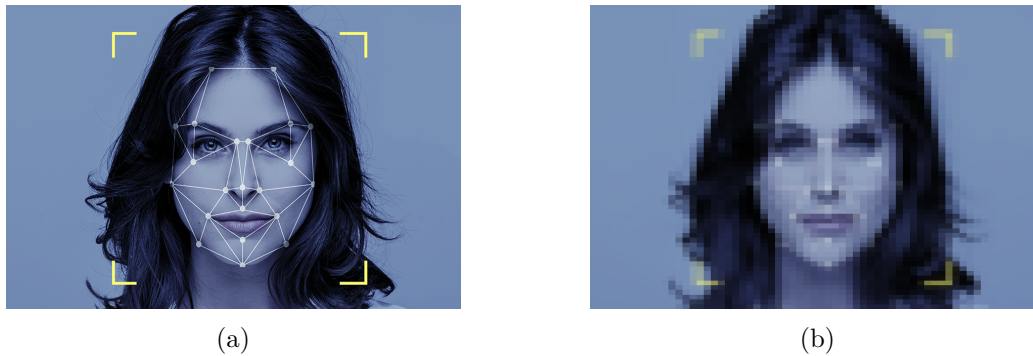


FIGURE 1.2 – Exemple de pré-traitement d'image (a) Initial Image. (b) Pixelated Image.

7.2 Apparence Active

Les algorithmes basés sur l'apparence active contiennent des informations statistiques sur une forme d'image et une variation de texture. Foulques et al. [Cootes, 2000] ont appliqué une analyse en composantes principales (PCA) pour générer un modèle statistique qui localise les points de référence sur un ensemble d'images d'apprentissage. Les points de référence sont utilisés pour entraîner le modèle à apprendre la translation entre le paramètre du modèle synthétisé et les images test. Pour trouver les paires d'image et les faire correspondre entre eux, l'erreur résiduelle du modèle est mesurée par rapport aux changements prédits des paramètres actuels du modèle ce qui conduit à un meilleur ajustement et à une meilleure reconnaissance [Zhao, 2003].

7.3 Support vector machines

Les machines à vecteurs de support utilisent un ensemble d'images d'apprentissage pour calculer l'hyperplan de séparation optimal. Guo et al. [Guo, 2000] ont appliqué cette méthode à la reconnaissance faciale à l'aide d'une classification arborescente binaire, où les images faciales sont classées de manière itérative comme appartenant à l'une des deux classes qui se propagent dans une structure arborescente binaire jusqu'à ce qu'une décision finale de classification puisse être prise.

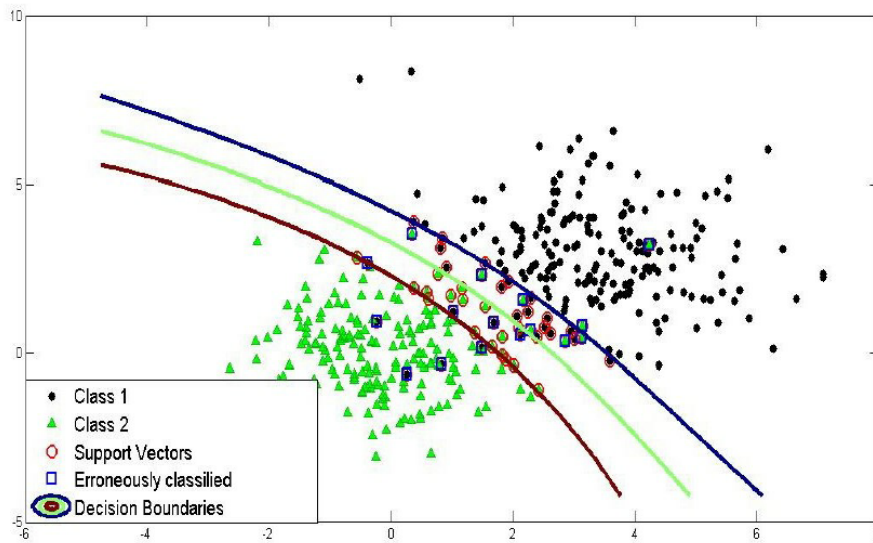


FIGURE 1.3 – SVM Classifier

7.4 Méthode Bayésienne

Les algorithmes du modèle bayésien montrent une mesure probabiliste de similarité dérivée d'une analyse bayésienne de la différence entre les images faciales. Le calcul des fonctions de probabilité utilisant les différences d'intensité d'image entre ensembles de données conduit à un score de similarité à partir de la règle de Bayes, qui peut ensuite être utilisé pour la classification de reconnaissance faciale [Moghaddam, 2000].

7.5 Neural Networks

Les réseaux de neurones assurent le traitement de l'information, qui est similaire à la manière dont l'information est traitée dans des systèmes biologiques tels que le cerveau humain. Leur principale force est la capacité d'apprendre à partir d'exemples, la tolérance aux pannes et la robustesse. Ils sont adaptés à la reconnaissance de visages qui présentent beaucoup de variation et nécessitent cependant peu de modifications de l'algorithme de reconnaissance. Lawrence [Lawrence, 1997] décrit comment entraîner un classificateur de réseau neuronal pour l'identification et la reconnaissance faciale.

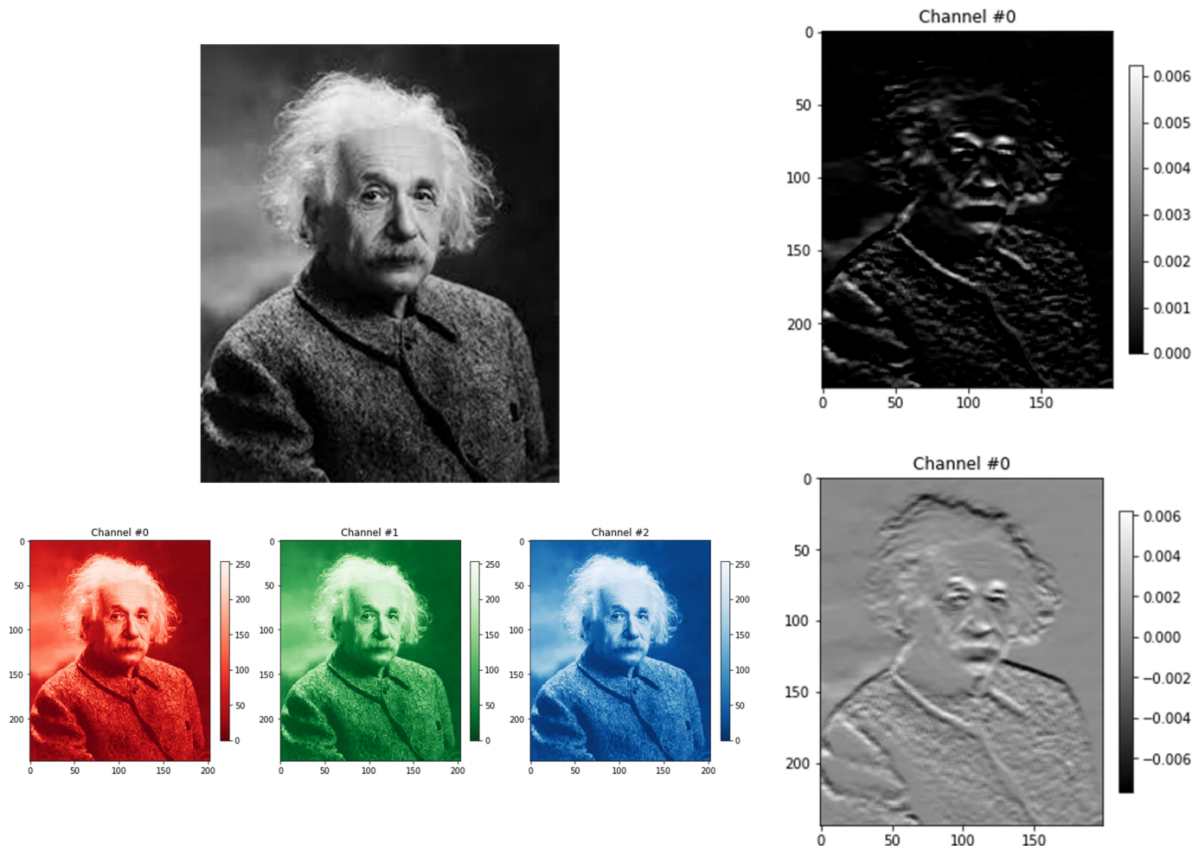


FIGURE 1.4 – Deep Convolutional Neural Networks for Quantum Computers | by Jonas Landman | Towards Data Science

7.6 Basés sur la texture

Les algorithmes basés sur la texture extraient la texture dans les images, en divisant le visage en plusieurs régions. Le modèle binaire local (LBP) est un exemple d'algorithmes basés sur la texture, où les caractéristiques LBP pondérées sont extraites pour générer un vecteur de différentes caractéristiques. Deux vecteurs de caractéristiques LBP sont appariés en appliquant une mesure de distance Chi-carré pondérée [Ahonen, 2006].

7.7 Basés sur les caractéristiques

Ces algorithmes extraient un ensemble de caractéristiques géométriques tel que la distance à partir des images faciales et utilisent ces caractéristiques comme une référence pour comparer les images. L'analyse des caractéristiques locales est un exemple d'algorithmes basés sur les caractéristiques [Penev, 1996].

Chapitre 2

Techniques de Reconnaissance Faciale

Objectifs

Dans ce chapitre nous allons définir les différentes techniques de reconnaissance faciale notamment les concepts mathématiques sous-jacent qui régissent la plupart des algorithmes décrit dans le chapitre précédent. A savoir les méthodes basées sur les manifoldes, valeurs et vecteurs propres qui traitent les images comme un système dynamique ou des approches basées sur les positions à angles multiples où le traçage des vidéos se fait sous différents angles.

Sommaire

1	Reconnaissance faciale	9
2	Eigen Face	9
3	Sub-espace facial	10
4	Approche basée sur des position à angles multiples	13
5	Traçage du visage à partir d'une video sous différents angles	14
6	Conclusion	14

1 Reconnaissance faciale

Parmi les six attributs biométriques pris en compte par [Heitmeyer, 2000], les traits du visage ont obtenu le plus grand score dans un système de contrôle de passeport lisibles à la machine douanière MRTD¹ sur la base d'un certain nombre de facteurs d'évaluation tels que l'inscription, le renouvellement, les exigences de la machine et le public. La Reconnaissance faciale comme étant la base des technologies biométriques, est devenue de plus en plus importante en raison des progrès rapides des dispositifs de capture d'images (caméras de surveillance, caméra des téléphones mobiles), de la disponibilité d'énormes quantités d'images sur le Web et des exigences accrues en matière de sécurité.

2 Eigen Face

La reconnaissance faciale est une mesure de similarité entre un visage inconnu et un ensemble de visages précédemment observés dans l'ensemble d'apprentissage. La similarité peut être établie en calculant la différence de distance entre les images. Lorsque cette différence est faible, le nouveau visage est considéré comme similaire à l'une des images de l'ensemble d'apprentissage, et il serait classé comme reconnu. Si la différence est importante, le nouveau visage sera considéré comme différent des images de l'ensemble d'apprentissage et il sera classé comme non reconnu. Supposons qu'un visage ne puisse être représenté que par 2 pixels et que l'ensemble d'apprentissage contienne quatre de ces images, a1, a2, b1 et b2, comme illustré à la [fig. 2.1].

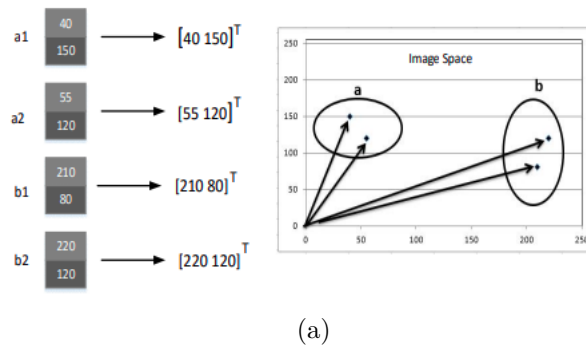


FIGURE 2.1 – A two pixel image mapped to a plane.

Le côté gauche de la [fig. 2.1] montre les images a1, a2, b1 et b2 avec les valeurs de pixel et les vecteurs transposés 1 x 2 correspondant. Le côté droit de la figure montre le vecteur transposé de chaque image mappée sur un plan. À partir des vecteurs d'image mappés, on peut voir que les images dont les valeurs de pixel sont proches les unes des autres sont également cartographiées sur des plans proches les uns des autres. Une nouvelle image représentée comme un point dans le plan peut alors être étiquetée comme "a" ou "b" ou ni l'un ni l'autre en calculant sa distance euclidienne des points respectifs dans "a" ou "b". La distance euclidienne entre les points peut être calculée par l'équation 1, où x et y sont des points dans le plan.

1. https://www.icao.int/meetings/tag-mrtd/tagmrtd22/tag-mrtd-22_wp24.pdf

$$distance = \sqrt{(\delta x)^2 + (\delta y)^2}$$

Tout comme une image à deux pixels peut être mappée dans un espace bidimensionnel (plan), des images plus volumineuses avec plus de pixels peuvent être mappées dans leurs dimensions respectives. Par exemple, une image 50 x 50 peut être mappée à 2500 dimensions dont la représentation est un point unique dans cet espace. Notez que chaque valeur de pixel représente l'une des dimensions. Un vecteur dans l'espace contient des éléments avec une correspondance un à un aux pixels de l'image. De plus, tout comme pour le plan, les images identiques dans une dimension supérieure sont plus proches les unes des autres et les images non similaires seront plus éloignées. Le calcul de la distance euclidienne en grande dimension implique de nombreuses soustractions entre l'image test et une image entraînée. Si chacune de ces différences contribue au bruit alors le nombre de bruit sera très élevé au final. En effet, la somme de toutes les différences au carré contribuera au bruit comparé à la quantité d'informations utiles.

Étant donné que le calcul de la différence entre les valeurs de pixel dans une dimension supérieure n'est pas pratique en raison du bruit de l'image, une réduction de dimensionnalité de l'espace de l'image doit être effectuée. La similarité entre les images sera alors basée sur la façon dont les points de l'image sont répartis dans cet espace de dimension réduite [Yambor, 2000].

3 Sub-espace facial

Bien que les technologies de reconnaissance faciale se sont considérablement améliorées et peuvent désormais être implémentées avec succès en "temps réel" pour des images et vidéos capturées dans des situations favorables, la reconnaissance faciale reste une entreprise difficile, en particulier pour les tâches sans contraintes où le point de vue, l'éclairage, l'expression, l'occlusion, et les traits faciaux peuvent varier considérablement. Cela peut être illustré d'un point de vue sub-spatial [fig. 2.2] ou par représentation de manifolds par exemple.



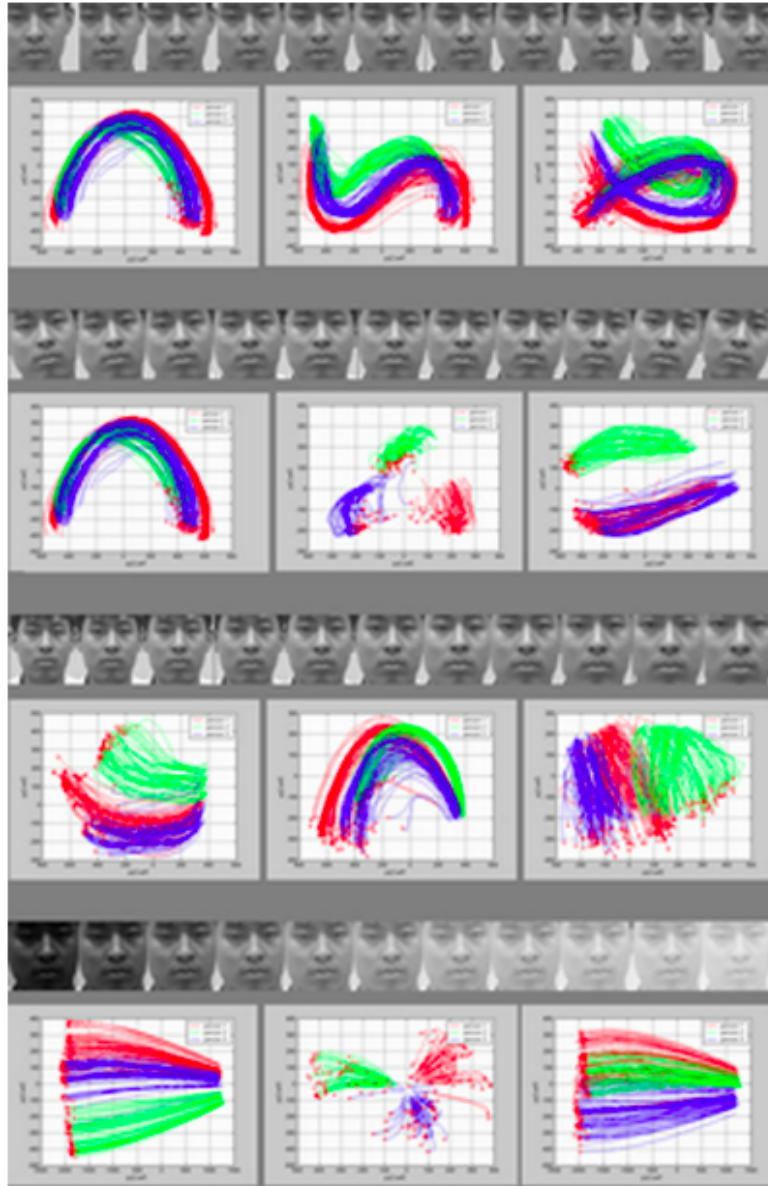
FIGURE 2.2 – Sub-espace Facial ou manifold (a) Face vs non face Manifolds. (b) Face Manifolds ou individus différents

Les techniques d'analyse de sous-espace pour la reconnaissance faciale sont basées sur le

fait qu'une classe de motifs d'intérêt, tels que le visage, réside dans un sous-espace de l'espace d'image d'entrée. Par exemple, une image $64 \times 64 \times 8$ bits avec 4096 pixels peut exprimer un grand nombre de classes de motifs, telles que des arbres, des maisons et des visages. Cependant, parmi les $256^{4096} > 10^{9864}$ "configurations" possibles, seule une infime partie correspond à des visages. Par conséquent, la représentation d'image basée sur les pixels est hautement redondante, et la dimensionnalité de cette représentation pourrait être considérablement réduite lorsqu'on s'intéresse seulement à certains traits du visage.

La manifold ou la distribution de tous les visages explique les variations de l'apparence faciale, tandis que la manifold non faciale représente tous les objets autres que les visages. Si on examine ces manifolds dans l'espace des images, on trouve des résultats non linéaires et non convexes [Bichsel, 1994], [Turk, 2001]. La [fig. 2.2\(a\)](#) illustre les manifolds faciales et non faciales et [fig. 2.2\(b\)](#) illustre les manifolds de deux individus dans la variété faciale entière. La détection du visage peut être considérée comme une tâche de distinction entre les manifolds faciale et non faciale dans l'espace des images (subwindow) et la reconnaissance faciale peut être considérée comme une tâche de distinction entre les visages de différents individus dans la manifold du visage.

La [fig. 2.3](#) démontre en outre la non-linéarité et la non-convexité des manifolds faciales dans un sous-espace PCA couvert par les trois premières composantes principales, où les tracés sont tirés de données réelles d'images faciales.



(a)

FIGURE 2.3 – Non linéarité et non convexité des manifolds faciales sous translation (du haut vers le bas), rotation, mise à l'échelle et transformations gamma

Chaque graphique représente les manifolds de trois individus (en trois couleurs). Les données consistent en 64 images faciales frontales pour chaque individu. Une transformée (transformée horizontale, rotation dans le plan, mise à l'échelle de taille et transformée gamma pour les 4 groupes, respectivement) est effectuée sur chaque image de visage avec 11 paramètres variant progressivement, produisant 11 images faciales transformées. Chacune des images transformées est recadrée pour ne contenir que la région du visage. Les 11 images faciales recadrées forment une séquence. Une courbe sur cette figure représente une telle séquence dans l'espace PCA, il y a donc 64 courbes pour chaque individu. L'espace PCA tridimensionnel (3D) est projeté sur trois espaces 2D différents (plans). On peut observer ainsi la non linéarité des trajectoires.

4 Approche basée sur des position à angles multiples

Le problème de la reconnaissance faciale dans des conditions générales d'observation (changement de pose) peut également être abordé à l'aide d'une formulation de Eigenspace. Il existe essentiellement deux façons d'aborder ce problème en utilisant l'infrastructure Eigenspace. Étant donné M individus placés sous différents angles différents C , on peut faire de la reconnaissance d'image et estimer la pose dans un eigensapce universel calculé à partir de la combinaison d'images MC .

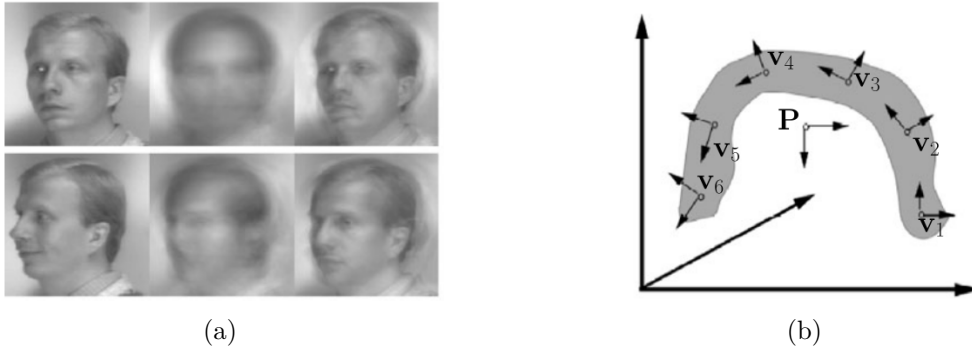


FIGURE 2.4 – Parametric versus view-based eigenspace methods. a Reconstructions of the input image (left) with parametric (middle) and view-based (right) eigenspaces. Top : training image ; bottom : novel (test) image. b Difference in the way the two approaches span the manifold

De cette façon, un seul eigenspace paramétrique permet d'encoder l'identité ainsi que la pose. Une telle approche, par exemple, a été utilisée par Murase et Nayar [Murase, 1995] pour la reconnaissance générale des objets 3D.

Alternativement, étant donné M individus placés sous différents angles C , nous pouvons construire un ensemble de eigenspaces C distincts, chacun capturant les variations des M individus placés sous un angle en commun. Dans cette approche basée sur les variations d'angles et plusieurs observateurs, la première étape consiste à déterminer l'emplacement et l'orientation de l'objet cible en sélectionnant l'eigenspace qui décrit le mieux l'image donnée en entrée. Ceci peut être accompli en calculant l'estimation de vraisemblance en utilisant les vecteurs propres de chaque espace d'angle afin d'en sélectionner le maximum.

5 Traçage du visage à partir d'une video sous différents angles

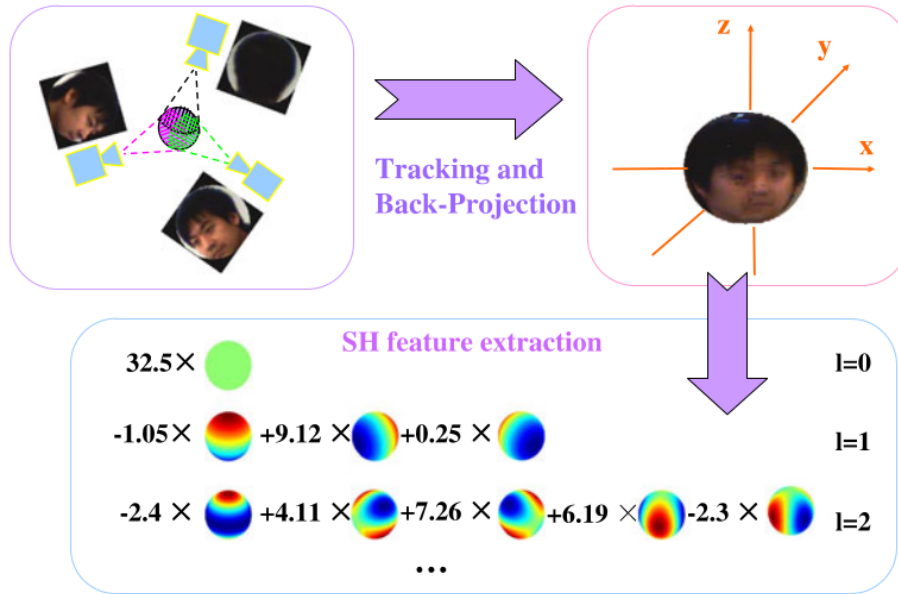


FIGURE 2.5 – Feature extraction. We first obtain the texture map of the human head on the surface of a spherical model through back-projection of multi-view images captured by the camera network, then represent it with spherical harmonics.

La fig. 2.5 montre un exemple de suivi sans pose pour une séquence vidéo à vues multiples. La séquence vidéo comporte 500 images. Le tracker est capable de suivre de manière stable toutes les images sans défaillance, malgré les mouvements de rotations, translation et recadrage de la tête. Parfois, la tête peut se trouver en dehors du champ de vision. Bien que les sujets ne subissent pas de mouvements aussi extrêmes dans les vidéos de surveillance réelles, cet exemple illustre clairement la fiabilité de cette méthode de suivi.

Les imprécisions parfois observées dans les cercles de délimitation sont principalement dus à la différence entre les sphères et l'exactitude des formes.

6 Conclusion

Les vidéos offrent plusieurs avantages comparés à la reconnaissance faciale sur des images statiques. En effet on a plus d'informations sur les mouvements, les angles et les variabilités d'expressions. Cependant, la disponibilité de la reconnaissance dynamique soulève plusieurs questions intéressantes, notamment sur les différentes façons possibles d'exploiter la corrélation temporelle pour un meilleur suivi facial, comment exploiter les indices comportementaux disponibles à partir de la vidéo et comment fusionner les multiples vues offertes par un réseau de caméras. Bien que plusieurs études suggèrent que les humains peuvent reconnaître facilement les visages dans des conditions non coopératives [OToole, 2002] telles qu'une mauvaise résolution, un mauvais éclairage ou des mouvements inattendus. Cette capacité a été difficile à décrire mathématiquement

et à reproduire algorithmiquement. Si ce phénomène peut être modélisé mathématiquement, il pourrait conduire à des systèmes de surveillance et une reconnaissance biométriques plus précise. Le rôle que joue l'émotion dans la reconnaissance des visages familiers, bien que décrite dans la littérature scientifique notamment en neurosciences [Roark, 2006], est encore une autre piste qui a été difficile à modéliser numériquement pour une réplique algorithmique.

Chapitre 3

Réalisation

Objectifs

Dans ce chapitre, on s'intéresse à mettre en place une application de reconnaissance faciale en implémentant les algorithmes de détection et d'alignement facial tels que décrit dans le projet OpenFace basé sur dlib et ses principales caractéristiques. Pour calculer les différentes représentations des visages, nous allons implémenter un réseau de neurones convolutifs avec Keras. Enfin, pour l'identification faciale nous allons construire un modèle SVM avec Scikit-Learn pour classifier les images.

Sommaire

1	Introduction	17
2	Reconnaissance faciale avec Keras, Dlib et OpenCV	17
2.1	Etape 1 : Détection	18
2.2	Etape 2 : CNN	18
2.3	Etape 3 : Vecteurs d'intégration	18
3	Architecture CNN et entraînement	18
3.1	Définition du CNN	18
3.2	Description de l'architecture CNN utilisée	19
4	Comprendre les incorporations de reconnaissance faciale d'apprentissage en profondeur	21
5	Alignement du visage	21
6	Embedding Vectors	22
7	Seuil de la distance	24
8	Reconnaissance Faciale	26
9	Visualisation des Données	27

1 Introduction

Les applications de reconnaissance faciale sont nombreuses et diverses, parmi elles on trouve par exemple : La vérification d'identité à l'aéroport, recherche d'une personne parmi les enregistrements de vidéo surveillance ou simplement déverrouiller un téléphone. Ce domaine a évolué rapidement ces dernières années, notamment depuis 2014 avec l'essor remarquable du deep learning.

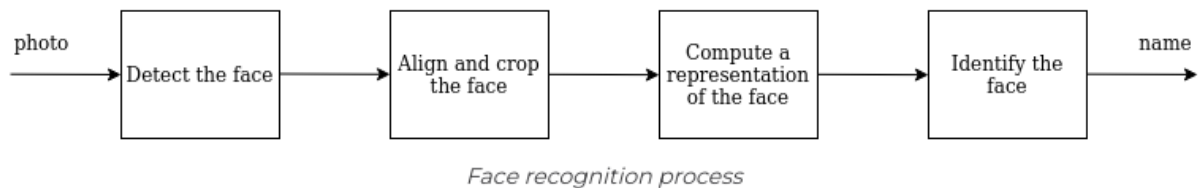


FIGURE 3.1 – Principes de la reconnaissance faciale

Le processus de reconnaissance faciale peut être divisé en 4 étapes :

- 1) **Détection du visage** : grâce à un apprentissage en profondeur qui a été formé pour détecter les visages.
- 2) **Alignement et encadrement** : en fonction de la position des points clés du visage (yeux, souris, nez, etc.).
- 3) **Calcul d'une représentation du visage** : avec un réseau de neurones convolutifs qui crée un encastrement du visage.
- 4) **Identification** : en comparant les encastresments avec ceux que nous avons dans la base de données.

2 Reconnaissance faciale avec Keras, Dlib et OpenCV

La reconnaissance faciale identifie les personnes sur des images ou des vidéos. En un mot, un système de reconnaissance faciale extrait les caractéristiques d'un visage en entrée et les compare aux caractéristiques des visages étiquetés dans la base de données. La comparaison est basée sur une métrique de similarité. Si la valeur de similarité est inférieure à un certain seuil, l'image d'entrée est étiquetée comme inconnue. La comparaison de deux images faciales pour valider l'identification est connue sous le nom de vérification faciale.

Dans ce chapitre on utilise un réseau de neurones convolutif (CNN) pour extraire les caractéristiques des images données en entrée. Le réseau suit l'approche décrite dans [Schroff, 2015] avec des modifications inspirées du projet OpenFace. Keras est utilisé pour implémenter CNN, Dlib et OpenCV pour aligner les visages sur les images d'entrée. Les performances de reconnaissance faciale sont évaluées sur un petit sous-ensemble de l'ensemble de données LFW, qu'il est possible

de remplacer par un ensemble de données personnalisé, par exemple, des images d'élèves inscrits dans l'université (pour une vérification biométrique à l'entrée).

On s'intéresse dans ce chapitre aux points suivants :

2.1 Etape 1 : Détection

- Détecter, transformer et recadrer les visages sur les images d'entrée. Cela garantit que les visages sont alignés avant de les introduire dans le CNN. Cette étape de pré-traitement est très importante pour les performances du réseau neuronal.

2.2 Etape 2 : CNN

- Utilisez le CNN pour extraire des représentations en 128 dimensions, ou incorporations, de visages à partir des images d'entrée alignées. Dans l'espace d'intégration, la distance euclidienne correspond directement à une mesure de la similarité des visages.

2.3 Etape 3 : Vecteurs d'intégration

- Comparez les vecteurs d'intégration d'entrée aux vecteurs d'intégration étiquetés dans une base de données. Ici, une machine à vecteurs de support (SVM) et un classificateur KNN, formés sur des vecteurs d'incorporation étiquetés, jouent le rôle d'une base de données. La reconnaissance faciale dans ce contexte signifie qu'il est nécessaire d'utiliser ces classificateurs pour prédire les étiquettes, c'est-à-dire les identifications des nouvelles entrées.

3 Architecture CNN et entraînement

3.1 Définition du CNN

Les réseaux de neurones convolutifs sont un genre spécifique de réseaux de neurones artificiels qui utilisent la convolution comme formule mathématique à la place de la multiplication matricielle dans au moins une de leurs couches. Ils sont spécifiquement conçus pour traiter les données en pixels et sont utilisés dans la reconnaissance et le traitement d'images.

Un réseau de neurone convolutif se compose d'une couche d'entrée, de couches cachées et d'une couche de sortie. Dans n'importe quel réseau de neurones dit "feed-forward", toutes les couches intermédiaires sont dites cachées car leurs entrées et sorties sont masquées par la fonction d'activation et la convolution finale. Dans un réseau neuronal convolutif, les couches cachées comprennent des couches qui effectuent des opérations de convolutions. Cela inclut généralement une couche qui effectue un produit scalaire du noyau de convolution avec la matrice d'entrée de la couche dont la fonction d'activation est généralement ReLU.

Lorsque le noyau de convolution glisse le long de la matrice d'entrée pour la couche, l'opération de convolution génère une carte de caractéristiques, qui à son tour contribue à l'entrée de la couche suivante.

Viennent ensuite d'autres couches telles que les couches de regroupement, les couches entièrement connectées et les couches de normalisation fig. 3.2.

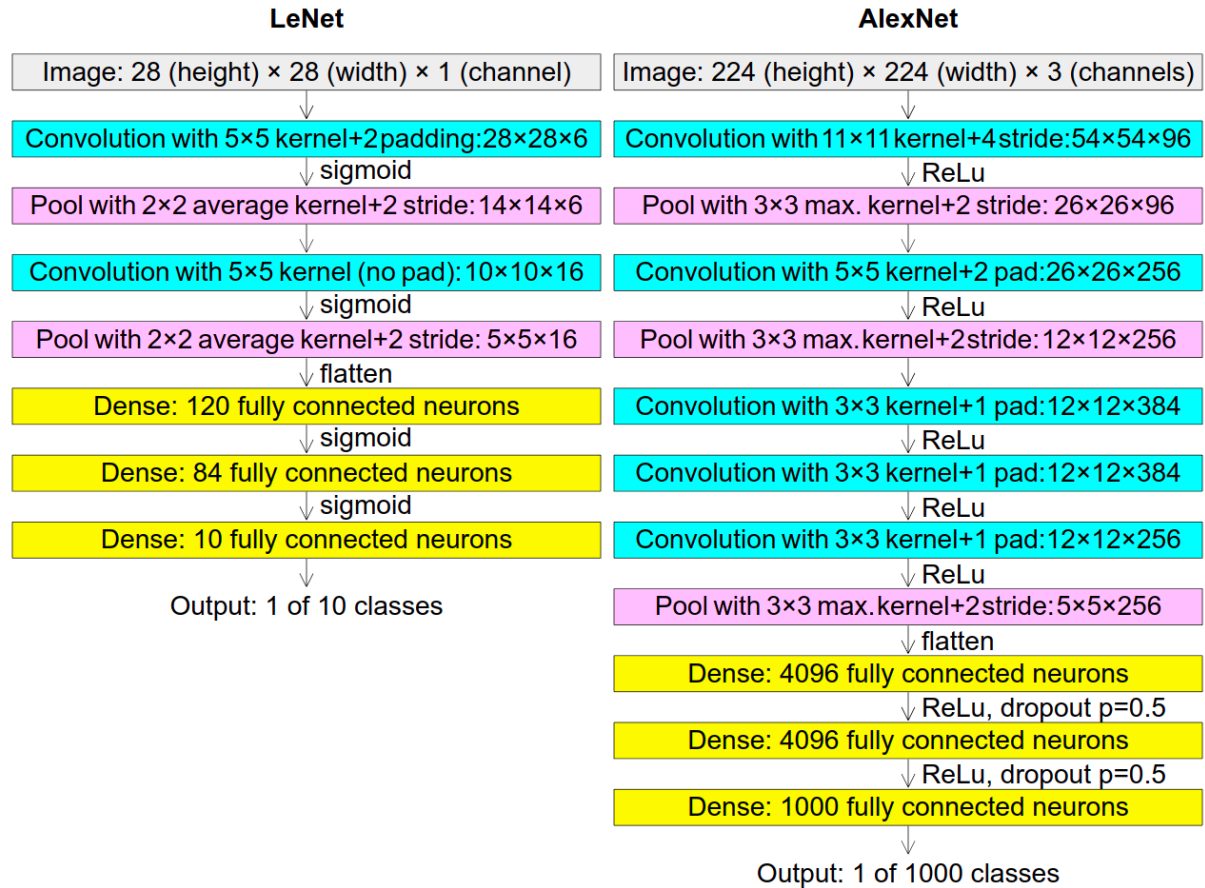


FIGURE 3.2 – Comparaison des convolutions LeNet et AlexNet

Référence de la fig. 3.2 voir note [wiki](https://en.wikipedia.org/wiki/File:Comparison_image_neural_networks.svg)¹

3.2 Description de l'architecture CNN utilisée

L'architecture CNN utilisée ici est une variante de l'architecture d'inception [Szegedy, 2014]. Plus précisément, il s'agit d'une variante de l'architecture CNN décrite dans [Schroff, 2015] et identifiée comme modèle nn4.small2 qu'on nomme ici model.cnn dans le projet OpenFace. Dans ce chapitre, on se charge d'une implémentations Keras de ce modèle dont la définition est tirée du projet Keras-OpenFace. Les détails de l'architecture ne sont pas détaillé ici, il est seulement utile de savoir qu'il existe une couche entièrement connectée avec 128 unités cachées suivie d'une couche de normalisation L2 au-dessus de la base convolutive. Ces deux couches supérieures sont

1. https://en.wikipedia.org/wiki/File:Comparison_image_neural_networks.svg

appelées la couche d'enrobage à partir de laquelle les vecteurs d'enrobage à 128 dimensions peuvent être obtenus. Le modèle complet est décrit dans `model.py`.

En python, une version Keras du modèle `model.cnn` peut être initialisée avec :

```
from model import create_model
model_cnn = create_model()
```

La formation de modèle vise à apprendre une incorporation $f(x)$ d'une image x telle que la distance $L2$ au carré entre tous les visages de la même identité est petite et la distance entre une paire de visages d'identités différentes est grande. Ceci peut être réalisé avec une *perte de triplet* L qui est minimisée lorsque la distance entre une image d'ancrage x_i^a et une image positive x_i^p (de la même identité) dans l'espace d'intégration est inférieure à la distance entre cette image d'ancrage et une image négative x_i^n (identité différente) d'au moins une marge α .

$$L = \sum_{i=1}^m [\|f(x)_i^a - f(x)_i^p\|_2^2 - \|f(x)_i^a - f(x)_i^n\|_2^2 + \alpha]_+ \quad (3.1)$$

Durant la phase d'entraînement, il est important de sélectionner les triplets dont les paires positives (x_i^a, x_i^p) et négatives (x_i^a, x_i^n) sont difficiles à discriminer. C'est-à-dire que leur différence de distance dans l'espace d'encastrement doit être inférieure à la marge α sinon, le réseau est incapable d'apprendre une intégration utile. Par conséquent, chaque itération de formation doit sélectionner un nouveau lot de triplets en fonction des intégrations apprises lors de l'itération précédente. En supposant qu'un générateur issu d'un appel à la fonction `triplet.generator()` est capable de générer des triplets sous ces contraintes alors le réseau peut être entraîné avec :

```
from data import triplet_generator
generator = triplet_generator()

model_cnn_trained.compile(loss=None, optimizer='adam')
model_cnn_trained.fit_generator(generator, epochs=10,
                                steps_per_epoch=100)
```

L'extrait de code ci-dessus doit simplement montrer comment configurer la phase train du modèle. Mais au lieu de réellement entraîner le modèle depuis le début, nous allons utiliser un modèle pré-entraîné car c'est moins coûteux et ne nécessite pas d'énormes ensembles de données pour obtenir de bonnes performances de généralisation. Par exemple, [Schroff, 2015] utilise un jeu de données de 200 millions d'images composé d'environ 8 millions d'identités.

Le projet OPENFACE fournit des modèles pré-formés qui ont été formés avec les ensembles de données de reconnaissance faciale publics tels que `FACESCUB` et `CASIA-WEBFACE`. Le projet Keras-OpenFace a converti les poids du modèle pré-formé `model.cnn.v1` en fichiers CSV qui ont

ensuite été convertis ici en un format binaire pouvant être chargé par Keras avec la fonction *load.weights* :

```
model_cnn_pretrained = create_model()  
model_cnn_pretrained.load_weights('weights/model.cnn.v1.h5')
```

4 Comprendre les incorporations de reconnaissance faciale d'apprentissage en profondeur

Un réseaux de neurone est entraîné pour deux buts principaux :

1. Accepter une seule image en entrée.
2. Et générer une classification/étiquette pour cette image.

Par contre l'apprentissage métrique en profondeur est différent. Au lieu d'essayer de produire une seule étiquette (ou coordonnées dans une image), nous produisons un vecteur caractéristique à valeur réelle.

Pour le réseau de reconnaissance faciale dlib, le vecteur caractéristiques de sortie est 128-d (c'est-à-dire une liste de 128 nombres à valeur réelle) qui est utilisé pour quantifier le visage. La formation du réseau se fait à l'aide de triplets.

En effet, la reconnaissance faciale via l'apprentissage métrique profond implique une "étape de formation triplet". Le triplet se compose de 3 images de visage uniques - 2 des 3 sont de la même personne. Le NN génère un vecteur 128-d pour chacune des 3 images de visage. Pour les 2 images de visage de la même personne, nous ajustons les poids du réseau de neurones pour rapprocher le vecteur via la métrique de distance.

5 Alignement du visage

Le modèle `model.cnn.v1` a été entraîné avec des images de visage alignées. Par conséquent, les images de visage du jeu de données personnalisé doivent également être alignées. Ici, nous utilisons Dlib pour la détection de visage et OpenCV pour la transformation et le recadrage d'image afin de produire des images de visage RVB 96X96 alignées. En utilisant l'utilitaire `AlignDlib` du projet `OpenFace`, c'est assez simple :

```
img = cv2.imread(path, 1)  
img1 = load_image(metadata[15].image_path())  
alignment = AlignDlib('models/landmarks.dat')  
bb1 = alignment.getLargestFaceBoundingBox(img1)  
alignment.align(96, img1, bb1, landmarkIndices=AlignDlib.  
    OUTER_EYES_AND_NOSE)
```



FIGURE 3.3 – Alignement du visages dans les photos

6 Embedding Vectors

Les vecteurs d'intégration peuvent maintenant être calculés en alimentant les images mises à l'échelle dans le réseau pré-formé.

Vérifions sur un seul exemple de triplet que la distance au carré L2 entre sa paire positive est inférieure à la distance entre sa paire négative.

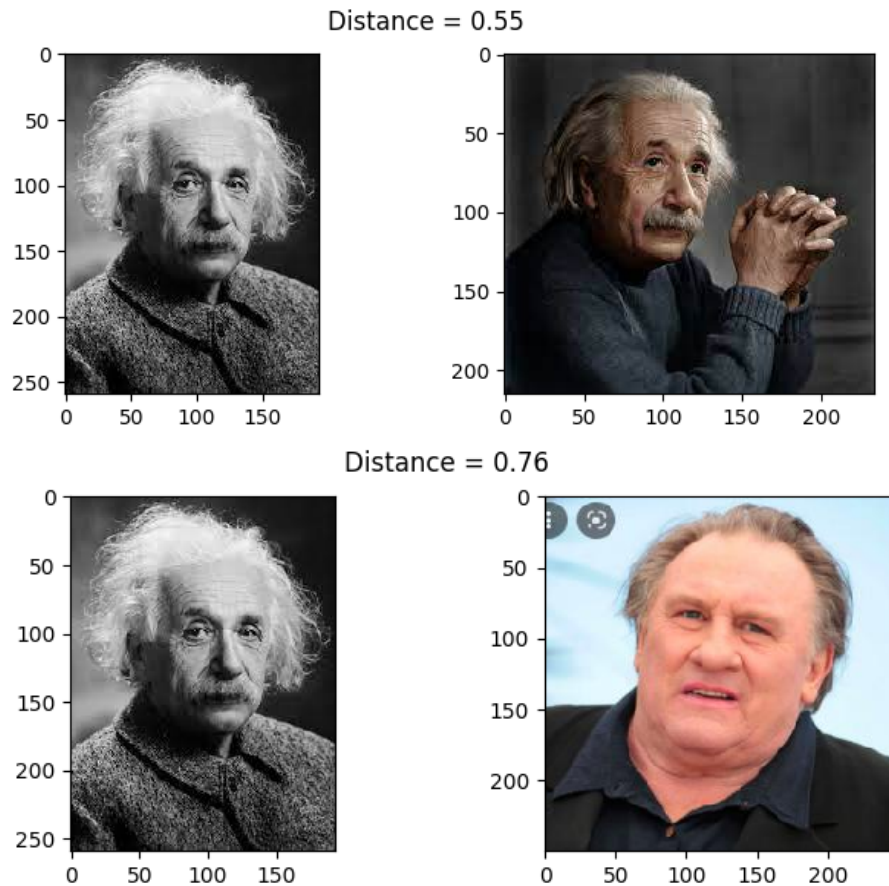


FIGURE 3.4 – Embedding vectors

L'analyse des données nous montre que la distance entre les deux images de Albert Einstein est inférieure à la distance entre une image de Albert Einstein et une autre image de la base de donnée ($0,55 < 0,76$). Mais nous ne savons toujours pas quel seuil de distance choisir et qui représente la meilleure limite décisionnelle entre une même identité et une identité différente.

En effet notre réseau quantifie les visages, en construisant l'incorporation 128-d (quantification) pour chacun.

À partir de là, l'idée générale est que nous allons ajuster les poids de notre réseau de neurones afin que les mesures à 128-d des deux Alberts Einstein soient plus proches l'une de l'autre et plus éloignées des mesures de Gérard Depardieu.

Notre architecture du réseau implémentée pour la reconnaissance faciale est basée sur ResNet-34 de l'article "Deep Residual Learning for Image Recognition de He et al.", mais avec moins de couches et le nombre de filtres réduit de moitié.

Le réseau lui-même a été formé par Davis King sur un ensemble de données d'approximativement 3 millions d'images. Sur l'ensemble de données LFW (Labeled Faces in the Wild), le réseau se compare à d'autres méthodes de pointe, atteignant une précision de 99,38 pourcent.

7 Seuil de la distance

La mesure du seuil de la distance répond à la problématique de déterminer à partir de quelle valeur de la distance on peut savoir si deux images sont identiques ou différentes.

Pour trouver la valeur optimale de τ , la vérification des visages doit être évaluée sur un intervalle de distances. À une valeur de seuil donnée, toutes les paires de vecteurs d'intégration possibles sont classées soit ayant la même identité ou soit ayant une identité différente et le tout comparé au résultat attendu. Puisque nous avons affaire à des classes asymétriques (beaucoup plus de paires négatives que de paires positives), nous utilisons le score F1 comme métrique d'évaluation au lieu de la précision.

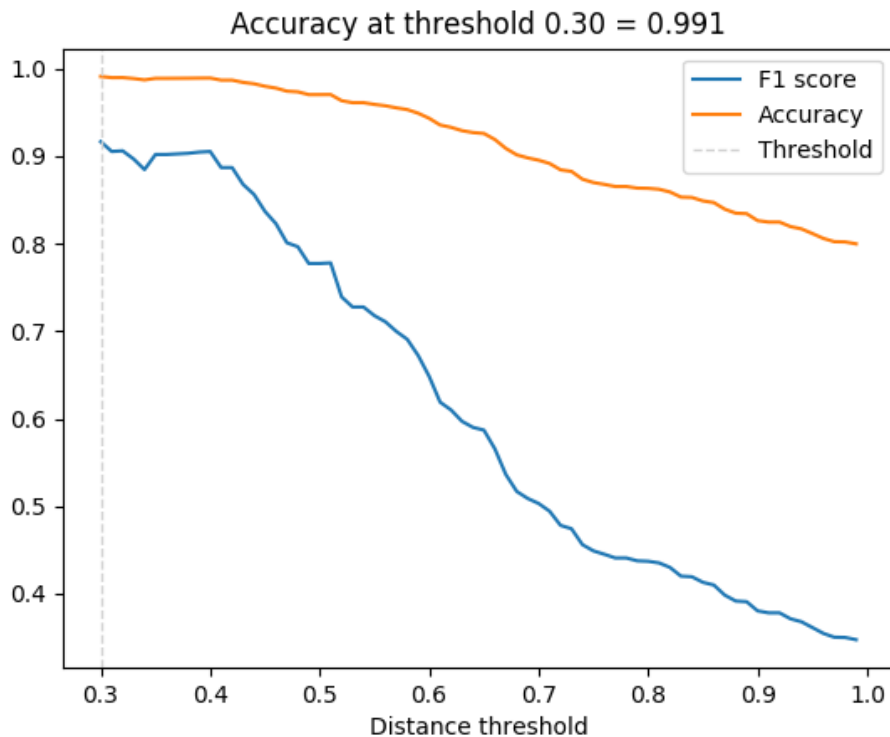


FIGURE 3.5 – Distance Threshold

Le score du modèle F1 représente le score du modèle en fonction de la précision et du score de rappel. Le F-score est une métrique de performance du modèle d'apprentissage automatique qui donne un poids égal à la fois à la précision et au rappel pour mesurer les performances en termes de précision, ce qui en fait une alternative idéale aux autres méthodes de mesure de la précision sans pour autant nous obliger à connaître le nombre total d'observations.

Quelques rappels de formules :

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

TABLE 3.1 – Accuracy equation

$$F1 = \frac{2}{Recall^{-1}+Precision^{-1}} = 2 \times \frac{Precision \times Recall}{Precision+Recall} = \frac{TP}{TP+\frac{1}{2} \times (FP+FN)}$$

TABLE 3.2 – Formule de calcul du score F1

$$Precision = \frac{TP}{TP+FP}$$

TABLE 3.3 – Formule de calcul de la précision

$$Recall = \frac{TP}{TP+FN}$$

TABLE 3.4 – Formule de calcul du rappel

L'algorithme de reconnaissance faciale ici présente une précision de 0.991 à $\tau = 0.30$. Ce n'est pas mal étant donné une ligne de base de 0.89 pour un classificateur qui prédit toujours une identité différente (il y a 450 paires positives et 3000 paires négatives), mais puisque nn4.small2.v1 est un modèle relativement petit, il est toujours inférieur à ce qui peut être atteint par les meilleurs modèles (> 0.99).

Les deux histogrammes suivants montrent les distributions de distance des paires positives et négatives et l'emplacement de la limite de décision. Il existe une nette séparation de ces distributions ce qui explique les performances discriminantes du réseau neuronal. On peut également repérer de fortes valeurs aberrantes dans la classe des paires positives.

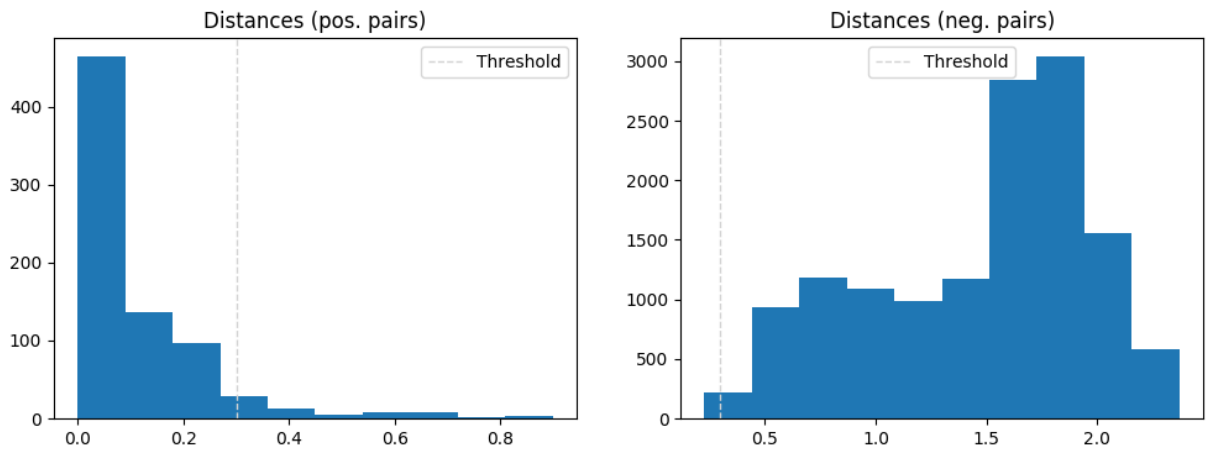


FIGURE 3.6 – Distance distributions of positive and negative pairs

8 Reconnaissance Faciale

Étant donné une estimation du seuil de distance τ , la reconnaissance faciale est maintenant aussi simple que de calculer les distances entre un vecteur d'intégration donné à l'entrée et tous les vecteurs d'intégration dans une base de données.

L'image donnée à l'entrée se voit attribuer une étiquette (c'est-à-dire une identité) d'entrée de la base de données avec la distance la plus petite si elle est inférieure à τ ou une étiquette inconnue dans le cas contraire.

Cette procédure peut également s'adapter à de grandes bases de données car elle peut être facilement parallélisée. Cet algorithme prend également en charge l'apprentissage ponctuel (au cas par cas), car l'ajout d'une nouvelle identité à l'entrée peut suffire à reconnaître de nouveaux exemples de la même identité.

Une approche plus robuste consiste à étiqueter les données à l'entrée en utilisant les entrées les mieux notées dans la base de données, ce qui représente essentiellement une classification KNN avec une métrique de distance euclidienne. Alternativement, une machine à vecteurs de support linéaire (SVM) peut être entraînée avec les entrées de la base de données et utilisée pour classer, c'est-à-dire identifier de nouvelles entrées. Pour la formation de ces classificateurs, nous utilisons 50 pourcent de l'ensemble de données et 50 pourcent pour l'évaluation.

```
KNN_accuracy = accuracy(Y_test, knn.predict(X_test))  
SVM_accuracy = accuracy(Y_test, svc.predict(X_test))
```

KNN.Accuracy = 0.96, SVM.Accuracy = 0.98

TABLE 3.5 – Resultat du calcul de l'accuracy

On voit que le classificateur KNN atteint une précision de 0.96 sur l'ensemble de test alors que le classificateur SVM atteint une précision de 0.98. Dans l'exemple suivant, on utilise le classificateur SVM pour tester la reconnaissance faciale :

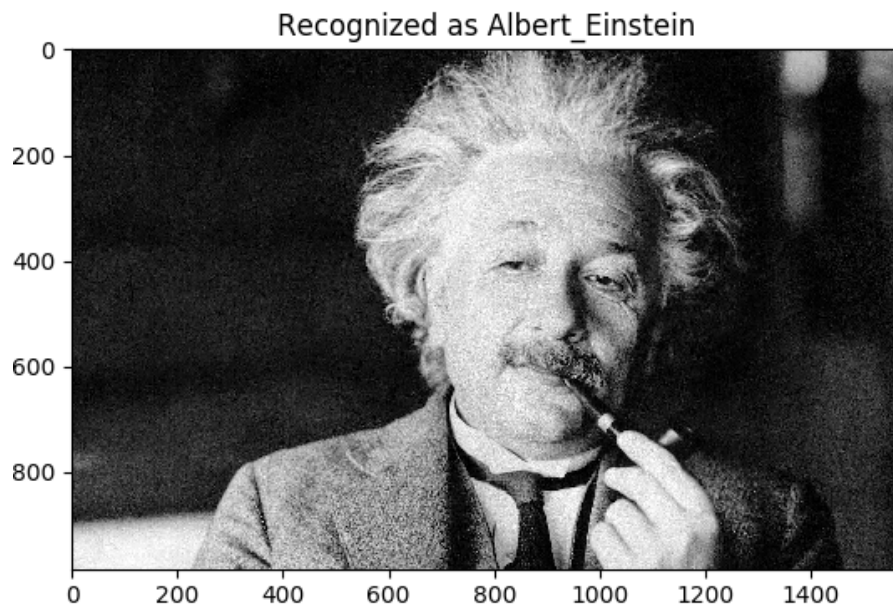


FIGURE 3.7 – Exemple de reconnaissance faciale par SVM

Bien que les deux algorithmes produisent des résultats satisfaisants en terme de score et précision. L'algorithme SVM offre un meilleur résultat avec une vitesse de classification nettement supérieur au KNN.

En effet, l'algorithme KNN utilise un système de vote pour déterminer à quelle classe appartient un objet non classé, en tenant compte de la classe de ses plus proches voisins dans l'espace de décision. L'algorithme SVM par contre est extrêmement rapide, capable de classer des images aériennes de 12 mégapixels en environ dix secondes (contrairement au KNN qui prend entre quarante et cinquante secondes).

Généralement, l'algorithme KNN classe avec précision ; cependant, il génère plusieurs petites erreurs de classification qui interfèrent avec l'image classifiée finale. Par contre, l'algorithme SVM classera parfois de manière erronée un objet volumineux qui interfère rarement avec l'image classifiée au final.

9 Visualisation des Données

Pour intégrer l'ensemble de données dans l'espace 2D et afficher les clusters d'identité on applique l'algorithme t-SNE aux vecteurs d'intégration à 128 dimensions.

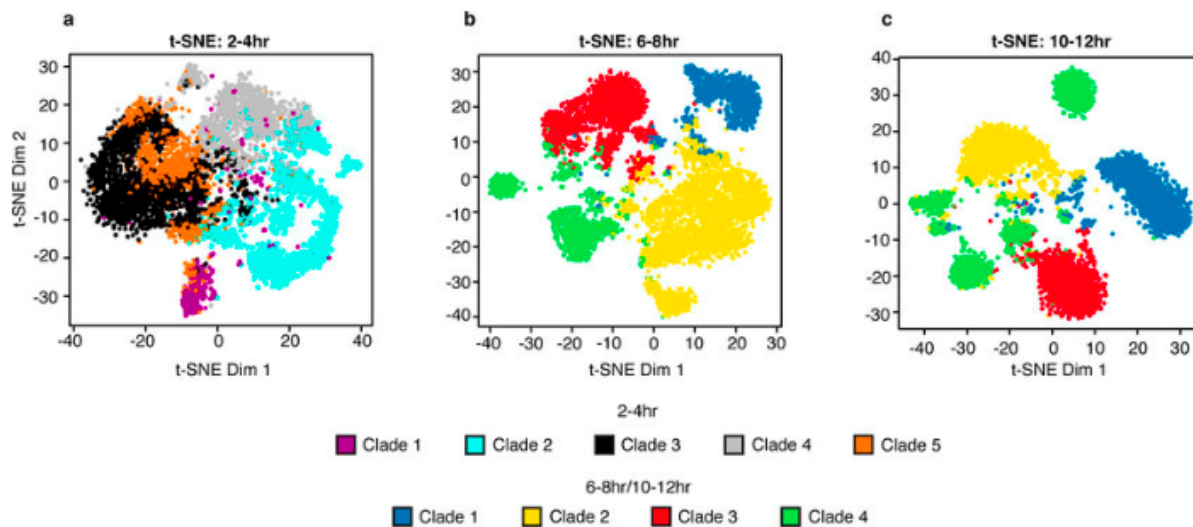


FIGURE 3.8 – Exemple de représentation t-SNE pour l’analyse des cellules à différents stades de développements

Référence voir note ²

« L’algorithme t-SNE (t-distributed stochastic neighbor embedding) est une technique de réduction de dimension pour la visualisation de données. Cet algorithme tente de trouver une configuration optimale selon un critère de théorie de l’information afin de conserver la proximité entre les points pendant la transformation : deux points qui sont proches (resp. éloignés) dans l’espace d’origine doivent être proches (resp. éloignés) dans l’espace de faible dimension. L’algorithme t-SNE se base sur une interprétation probabiliste des proximités. Une distribution de probabilité est définie sur les paires de points de l’espace d’origine de telle sorte que des points proches l’un de l’autre ont une forte probabilité d’être choisis tandis que des points éloignés ont une faible probabilité d’être sélectionnés. Une distribution de probabilité est également définie de la même manière pour l’espace de visualisation. L’algorithme t-SNE consiste à faire concorder les deux densités de probabilité, en minimisant la divergence de Kullback-Leibler entre les deux distributions par rapport à l’emplacement des points sur la carte. ³. »

Dans l’exemple illustré suivant on voit qu’à l’exception de quelques valeurs aberrantes, les clusters d’identité sont bien séparés et on peut distinguer différents groupes selon leurs positions sur le graph.

2. https://www.researchgate.net/publication/323781719_The_cis-regulatory_dynamics_of_embryonic_development_at_single-cell_resolution

3. https://fr.wikipedia.org/wiki/Algorithme_t-SNE

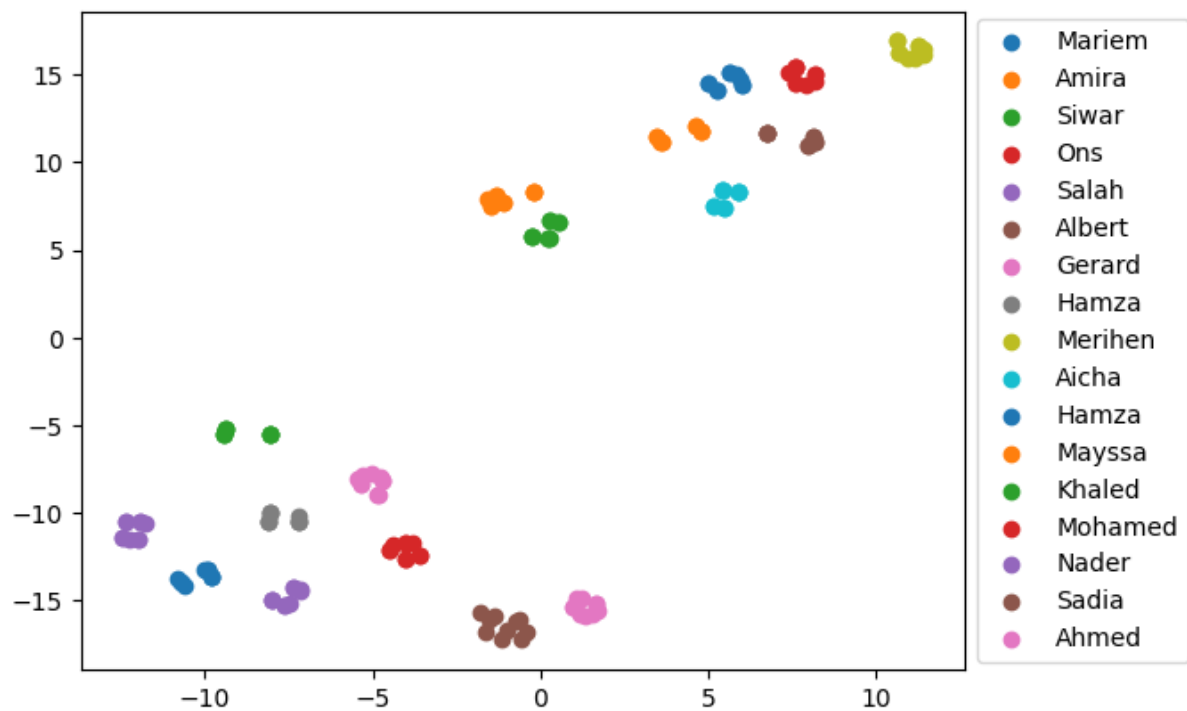


FIGURE 3.9 – t-distributed Stochastic Neighbor Embedding

Dans notre cas plus les personnes se ressemblent et ont des traits faciaux en communs et plus ils seront proches dans l'espace du graph [fig. 3.9](#). En effet on distingue deux clusters principaux un de droite et l'autre de gauche : les étudiants masculins sont dans le cluster de gauche alors que les étudiantes de sexe féminin sont dans le cluster de droite. Bien qu'une grande distance sépare les deux groupes, la différence des traits n'est pas aussi aberrante pour les personnes se trouvant au sein du même cluster (que ce soit cluster 1 ou cluster 2).

L'objectif principal de t-SNE est de projeter des points multidimensionnels sur des plans bidimensionnels ou tridimensionnels de sorte que si deux points sont proches de l'espace initial à grande dimension, ils restent proches dans la projection résultante. Si les points sont éloignés les uns des autres, ils sont forcement éloignés dans l'espace à faible dimension vers lequel ils projettent.

Il s'agit essentiellement d'un algorithme d'optimisation qui utilise méthode de descente en gradient stochastique avec une fonction de coût basée sur la divergence Kullback – Leibler qui est une mesure couramment utilisée pour estimer la différence entre deux distributions de données.

Chapitre 4

Mise en place de l'application de Reconnaissance Faciale

Objectifs

Dans ce chapitre nous allons continuer la mise en place de l'application de reconnaissance faciale en implémentant une analyse statistique des données avec Microsoft Power BI. On va également revenir sur les formules mathématiques de certains algorithmes tels que EigenFaces, FisherFaces et Local Binary Patterns Histograms LBPH afin de pouvoir apporter certaines modifications qui nous permettraient de les adapter dans notre algorithme si nécessaire.

Sommaire

1	Planification	31
2	Déscription de l'architecture de l'application	31
3	Live Camera Recognition	31
3.1	Eigenfaces	32
3.2	Fisherfaces	34
3.3	Local Binary Patterns Histograms	36
4	CNN Face Recognition	40
4.1	Description du mode de fonctionnement de keras	40
5	Analyse des données	43
6	Visualisation des données avec PowerBI	43
6.1	Représentation par spécialité	44
6.2	Représentation par matière fréquentée	45
7	En Résumé	45

1 Planification

La planification du projet s'est fait selon les principales étapes de management de projet décrite dans le Gantt chart qu'on peut trouver ci-joint en annexe.

2 Description de l'architecture de l'application

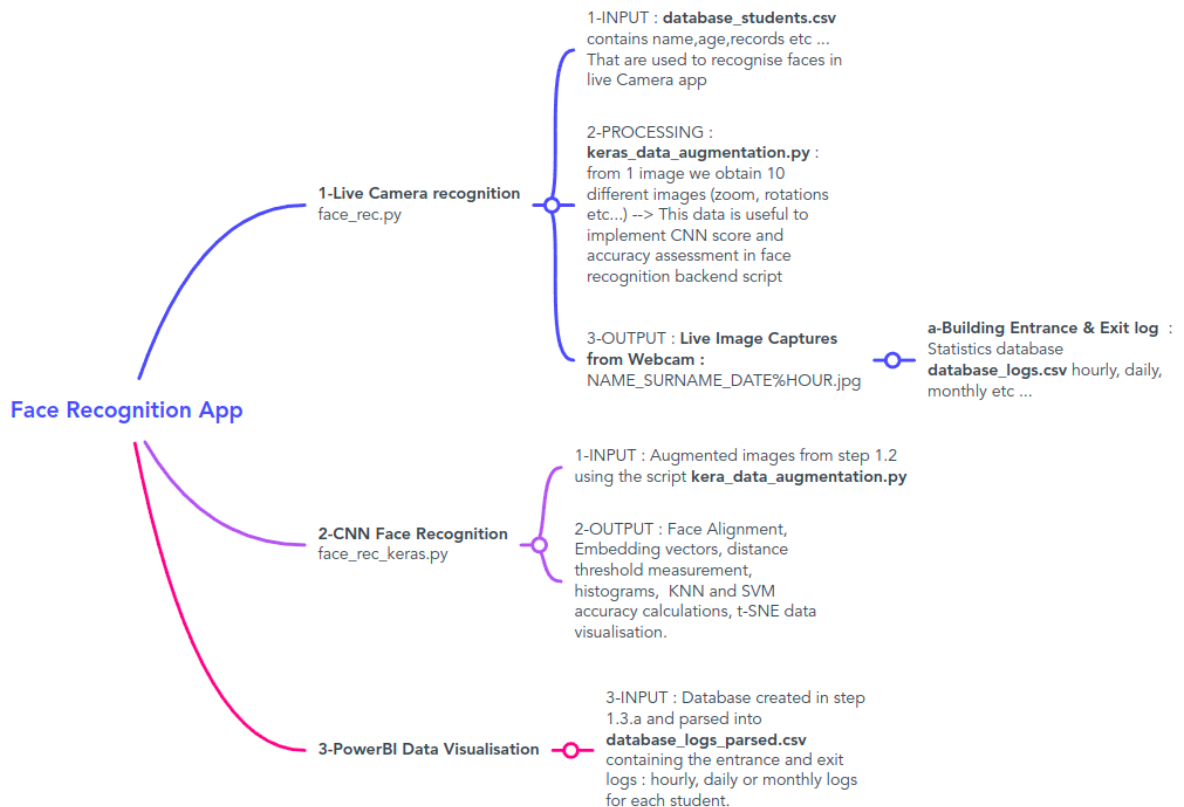


FIGURE 4.1 – Schéma résumant la structure de l'application mise en place pour la reconnaissance faciale

L'application mise en place dans ce projet se divise en trois grandes parties :

3 Live Camera Recognition

Dans la première partie on implémente une reconnaissance faciale en direct avec un système de surveillance en temps réel (caméra placé à l'entrée du bâtiment). Pour cela on utilise les bibliothèques face-recognition et OpenCv2.

Les données utilisées à l'entrée sont décrites dans database-students.csv et comprennent le nom, prénom, age, spécialité etc ... de chaque étudiant.

Ces données sont analysées avec l'algorithme OpenCv2 d'une part et face-recognition d'autre

part pour obtenir une reconnaissance faciale sur une caméra en marche.

Il est à noter que parmi les inconvénients de OpenCv2 est qu'il utilise un moyen de calcul qui devient de moins en moins rapide si on utilise une large base de donnée dans la phase d'entraînement.

Néanmoins cela reste un algorithme assez simple et rapide à implémenter avec diverses modules et méthodes de calcul pour effectuer la reconnaissance faciale :

1. EigenFace Recognizer qui peut être implémenté avec `cv2.face.createEigenFaceRecognizer()`
2. FisherFace Recognizer qui peut être implémenté avec `cv2.face.createFisherFaceRecognizer()`
3. Local Binary Patterns Histogram (LBPH) qui peut être implémenté avec `cv2.face.LBPHFisherFaceRecognizer()`

Dans notre cas on a choisi d'utiliser l'algorithme LBPH car il est plus robuste pour les transformations des images sur une echelle de gris.

3.1 Eigenfaces

L'algorithme Eigenfaces a été développé afin de répondre aux problèmes de représenter les images à N dimensions.

3.1.1 Exemple

Les images bidimensionnelles en niveaux de gris $p \times q$ s'étendent sur un espace vectoriel de dimension $m = pq$, de sorte qu'une image de 100×100 pixels se trouve déjà dans un espace d'image de 10 000 dimensions. La question qui se pose donc est la suivante : toutes les dimensions sont-elles utiles sans exception ? Pour résoudre cette problématique on a besoin d'analyser plus en détails les variations des données et leurs composantes .

L'analyse en composantes principales (ACP) a été proposée indépendamment par Karl Pearson (1901) et Harold Hotelling (1933) pour transformer un ensemble de variables éventuellement corrélées en un ensemble plus petit de variables non corrélées. L'idée est qu'un ensemble de données de plus grande dimension est souvent décrit par des variables corrélées et que, par conséquent, seules quelques dimensions significatives représentent la plupart des informations.

3.1.2 Description algorithmique de la méthode EigenFaces

Soit l'ensemble $X = \{x_1, x_2, \dots, x_n\}$ un vecteur aléatoire avec des observations $x_i \in R^d$.

1. Calcul de la moyenne μ :

$$\mu = \frac{1}{n} \sum_{i=1}^m x_i \quad (4.1)$$

2. Calcul de la matrice de covariance S :

$$S = \frac{1}{n} \sum_{i=1}^m (x_i - \mu)(x_i - \mu)^T \quad (4.2)$$

3. Calcul des eigenvalues λ_i et eigenvectors ν_i de S :

$$S\nu_i = \lambda_i\nu_i, i = 1, 2, \dots, n \quad (4.3)$$

4. Classification des vecteurs propres (eigenvectors) de manière descendante selon leurs valeurs propre (eigenvalues). Les κ composantes principales sont les vecteurs propres correspondant aux κ plus grandes valeurs propres. Les κ composantes principales du vecteur observé x sont alors données par :

$$y = W^T(x - \mu) \quad (4.4)$$

où $W = (\nu_1, \nu_2, \dots, \nu_k)$.

La reconstruction par PCA est donnée par :

$$x = Wy + \mu \quad (4.5)$$

où $W = (\nu_1, \nu_2, \dots, \nu_k)$.

La méthode Eigenfaces effectue ensuite la reconnaissance faciale par :

- Projection de tous les échantillons d'apprentissage sur l'espace PCA.
- Projection de l'image test sur l'espace PCA.
- Recherche du voisin le plus proche entre les images d'entraînement projetées et l'image test.

Il reste cependant un problème à résoudre. Dans le cas où l'on reçoit 400 images de 100×100 pixels. L'analyse en composantes principales résout la matrice de covariance $S = XX^T$, où la taille est $size(X) = 10000 \times 400$ dans notre exemple.

On se retrouve avec une matrice 10000×10000 , soit environ 0,8 Go. Résoudre ce problème n'est pas faisable, mais on peut s'y prendre autrement : On sait qu'une matrice $M \times N$ avec $M > N$ ne peut avoir que $N - 1$ valeurs propres non nulles. Il est donc possible d'utiliser à la place la décomposition en valeurs propres $S = X^T X$ de taille $N \times N$:

$$X^T X \nu_i = \lambda_i \nu_i \quad (4.6)$$

Et obtenir les vecteurs propres originaux de $S = XX^T$ avec une multiplication à gauche de la matrice de données :

$$XX^T(X\nu_i) = \lambda_i(X\nu_i) \quad (4.7)$$

Les vecteurs propres résultants sont orthogonaux, pour obtenir des vecteurs propres orthonormés, ils doivent être normalisés à la longueur unitaire.

3.2 Fisherfaces

L'analyse en composantes principales (ACP) qui est la méthode principale utilisée dans Eigenfaces, se base principalement sur la manière de trouver la meilleure combinaison linéaire qui maximise la variance totale dans les données.

Bien que ce soit un moyen puissant de représenter les données, cela ne prend pas en compte les classes et de nombreuses informations importantes peuvent être perdues lors de l'élimination de certains composants.

3.2.1 Exemple

Si on prend l'exemple d'un cas où la variance des données est générée par une source externe (exemple lumière). Les composants identifiés par une PCA ne contiennent pas nécessairement d'informations utiles pour discriminer les résultats, de sorte que les échantillons projetés sont confondus et la classification devient impossible. (cf [PCA with GNU Octave](#)¹).

L'analyse par discrimination linéaire effectue la réduction de la dimensionnalité en se basant sur une classe. Cette méthode a été inventée par R. A. Fisher qu'il a utilisé avec succès pour classer les fleurs dans son article de 1936 (Utilisation de mesures multiples pour répondre à des problèmes taxonomiques).

Afin de trouver la combinaison de caractéristiques qui sépare le mieux les classes, l'analyse par discrimination linéaire maximise le rapport de dispersion inter et intra classes, au lieu de maximiser la dispersion globale.

L'idée étant simple : les mêmes classes doivent être étroitement regroupées, tandis que les classes complètement différentes doivent se trouver aussi éloignées que possible les unes des autres dans les projection de dimensions inférieures.

Cette méthode a aussi été expérimentée pour la reconnaissance faciale notamment dans les articles de Belhumeur, Hespanha et Kriegman [Belhumeur, 1997].

3.2.2 Description algorithmique de la méthode Fisherfaces

Soit X un vecteur aléatoire avec un échantillon de c classes :

1. https://www.bytefish.de/blog/pca_lda_with_gnu_octave.html

$$X = \{X_1, X_2, \dots, X_c\} \quad (4.8)$$

$$X_i = \{x_1, x_2, \dots, x_n\} \quad (4.9)$$

Les Matrices de diffusion S_B et S_W sont calculé comme suit :

$$S_B = \sum_{i=1}^c N_i (\mu_i - \mu)(\mu_i - \mu)^T \quad (4.10)$$

$$S_W = \sum_{i=1}^c \sum_{x_j \in X_i} (x_j - \mu_i)(x_j - \mu_i)^T \quad (4.11)$$

où μ représente la moyenne totale :

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (4.12)$$

Et μ_i représente la moyenne de la classe $i \in \{1, \dots, c\}$:

$$\mu_i = \frac{1}{|X_i|} \sum_{x_j \in X_i} x_j \quad (4.13)$$

L'algorithme classique de Fisher recherche maintenant une projection W , qui maximise le critère de séparation des classes :

$$W_{opt} = \underset{W}{argmax} \frac{|W^T S_B W|}{|W^T S_W W|} \quad (4.14)$$

Une solution à ce problème d'optimisation consiste à résoudre le Problème Général des eigen values :

$$S_B \nu_i = \lambda_i S_W \nu_i \quad (4.15)$$

$$S_W^{-1} S_B \nu_i = \lambda_i \nu_i \quad (4.16)$$

Il reste un problème à résoudre : le rang de S_W est au plus $(N - c)$, avec N échantillons et c classes. Dans les problèmes de reconnaissance de formes, le nombre d'échantillons N est presque toujours inférieur à la dimension des données d'entrée (nombre de pixels), de sorte que la matrice de dispersion S_W devient singulière comme dans l'exemple de [Raudys, 1991]. Dans [Belhumeur, 1997], cela a été résolu en effectuant une analyse en composantes principales sur les données et en projetant les échantillons dans l'espace de dimension $(N - c)$. Une analyse par

discrimination linéaire a ensuite été effectuée sur les données réduites, car S_W ne présente plus de singularité.

Le problème d'optimisation peut être défini comme suit :

$$W_{pca} = \underset{W}{\operatorname{argmax}} |W^T S^T W| \quad (4.17)$$

$$W_{fld} = \underset{W}{\operatorname{argmax}} \frac{|W^T W_{pca}^T S_B W_{pca} W|}{|W^T W_{pca}^T S_W W_{pca} W|} \quad (4.18)$$

La matrice de transformation W qui projette un échantillon dans un espace de dimension $(c - 1)$ est donné par :

$$W = W_{fld}^T W_{pca}^T \quad (4.19)$$

3.3 Local Binary Patterns Histograms

Eigenfaces et Fisherfaces adoptent une approche holistique pour la reconnaissance faciale. On traite les données comme un vecteur dans un espace d'image de grande dimension. Nous savons que travailler dans une dimension élevée n'est pas avantageux, donc on essaye d'identifier un sous-espace de dimension inférieure, où probablement des informations utiles sont toujours préservées. L'approche Eigenfaces maximise la dispersion totale, ce qui peut entraîner des problèmes si la variance provient d'une source externe, car les composants qui ont une variance maximale sur l'ensemble des classes ne sont pas nécessairement ceux utiles pour la classification (cf [PCA with GNU Octave²](#)). Ainsi, pour préserver certaines informations utiles, nous avons appliqué une analyse par discrimination linéaire avant d'optimiser l'algorithme (comme décrit dans la méthode Fisherfaces). La méthode Fisherfaces a très bien fonctionné... du moins pour le scénario initial mis en place pour ce modèle.

3.3.1 Exemple

Dans la réalité certaines conditions ne sont pas parfaite pour une reconnaissance optimale. On ne peut tout simplement pas garantir des paramètres d'éclairage parfaits dans toutes les images ou avoir 10 images différentes d'une seule personne.

En effet si on a qu'une seule image pour chaque personne, les estimations de covariance dans le sous-espace correspondant peuvent être horriblement fausses, tout comme la reconnaissance faciale.

Voici par exemple les taux de reconnaissance de rang 1 de la méthode Eigenfaces et Fisherfaces

2. https://www.bytefish.de/blog/pca_lda_with_gnu_octave.html

sur AT&T Facedatabase, qui est une base de données d'images assez simple :

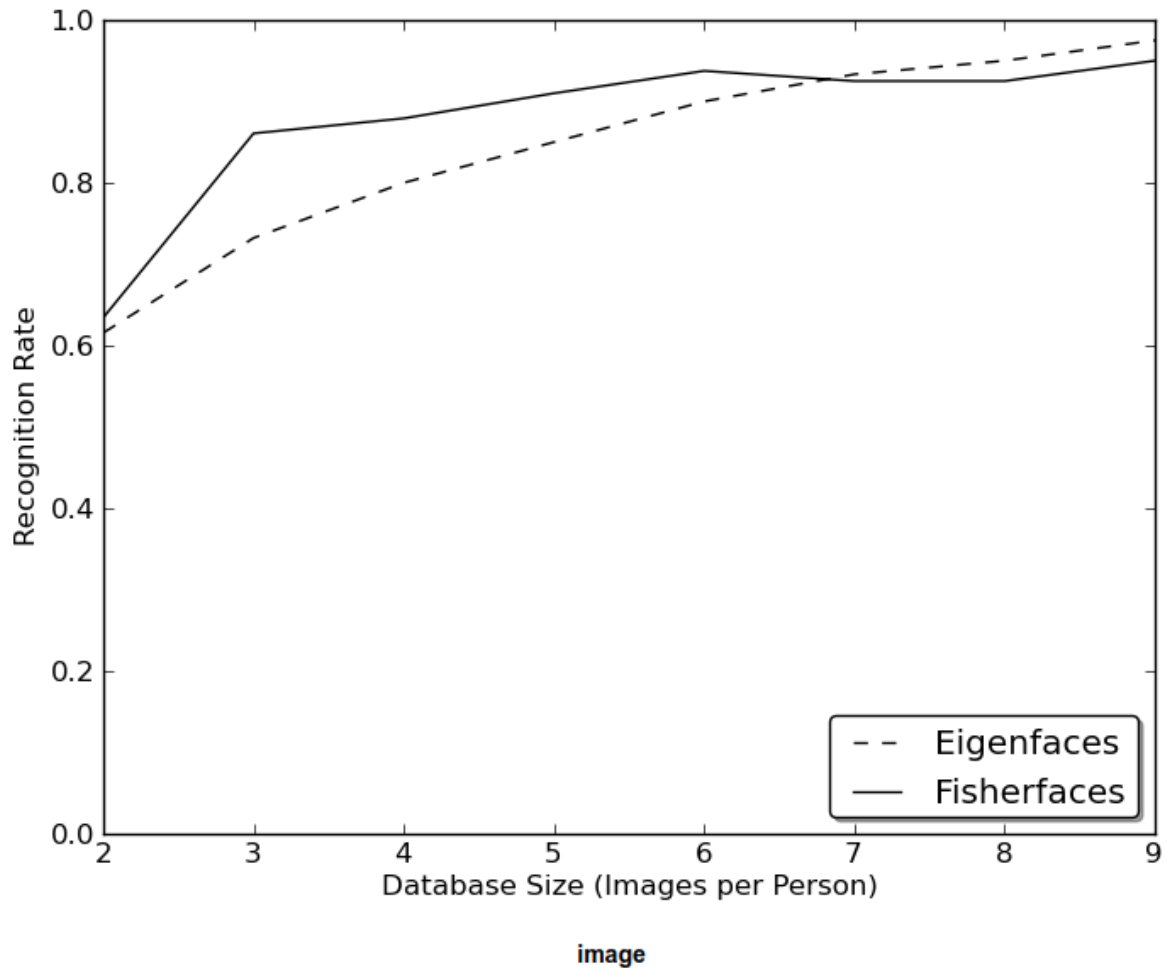


FIGURE 4.2 – Représentation des taux de reconnaissance pour les méthodes eigenface et fisherface

On voit que pour avoir de bon taux de reconnaissance il faut au moins disposer de 8 images par personne sinon la méthode fisherface ne fonctionne plus. Les résultats représentés ici sont validés par croisement et obtenu avec le framework [facerec](https://github.com/bytefish/facerec)³.

Ainsi, certaines recherches se sont concentrées sur l'extraction des caractéristiques locales dans les images. L'idée n'était pas de considérer l'image entière comme un vecteur de haute dimension, mais de ne décrire que les caractéristiques locales d'un objet. Les fonctionnalités extraites de cette façon auront ainsi une dimension faible implicitement. Cette méthode n'est efficace que dans le cas des variations d'illuminations mais cela devient plus compliqué si d'autres facteurs sont impliqués comme l'échelle, les translations ou les rotations des images. Similaire à SIFT, la méthode des motifs binaires locaux tire ses origines dans l'analyse des textures 2D.

3. <https://github.com/bytefish/facerec>

L'idée de base est de résumer la structure locale d'une image en comparant chaque pixel avec son voisinage. Prendre un pixel comme centre et estimer le seuil de ses voisins par rapport à ce pixel. Si l'intensité du pixel central est plus grande ou égale à son proche voisin, on le marque avec 1 ou 0 sinon.

Si on prends 8 pixels environnant on obtient 2^8 combinaisons possibles c'est ce qui s'appelle les motifs binaires locaux (local binary patterns) ou codes *LBP*.

Le premier opérateur LBP décrit dans la littérature utilise un voisinage fixe de 3×3 comme suit :

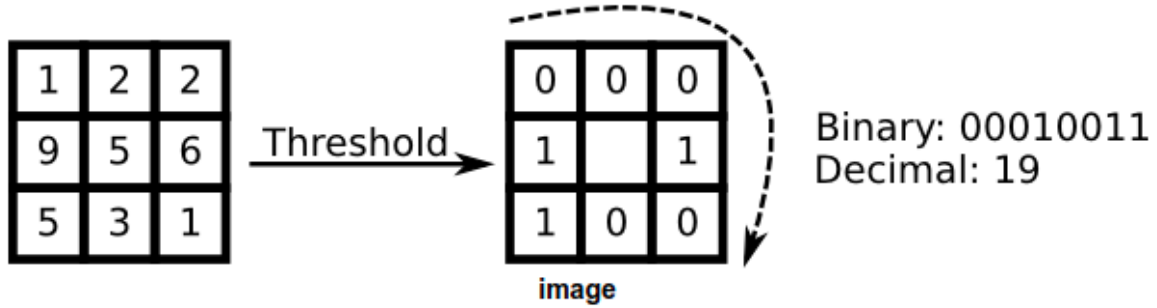


FIGURE 4.3 – Opérateur LBP avec un voisinage 3×3

3.3.2 Description algorithmique de la méthode LBPH

Une description plus formelle de l'opérateur peut être représentée par :

$$LBP_{(x_c, y_c)} = \sum_{p=0}^{p-1} 2^p s(i_p - i_c) \quad (4.20)$$

avec (x_c, y_c) comme pixel central avec une intensité i_c , et i_n l'intensité du pixel avoisinant. s représente la fonction signal définie par :

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{else} \end{cases}$$

Cette description nous permet de capturer des détails très fins dans les images. L'article [Ahonen, 2006] a introduit une nouvelle idée qui consiste à aligner un nombre arbitraire de voisins sur un cercle de rayon variable qui permet de capturer le voisinage de manière suivante :

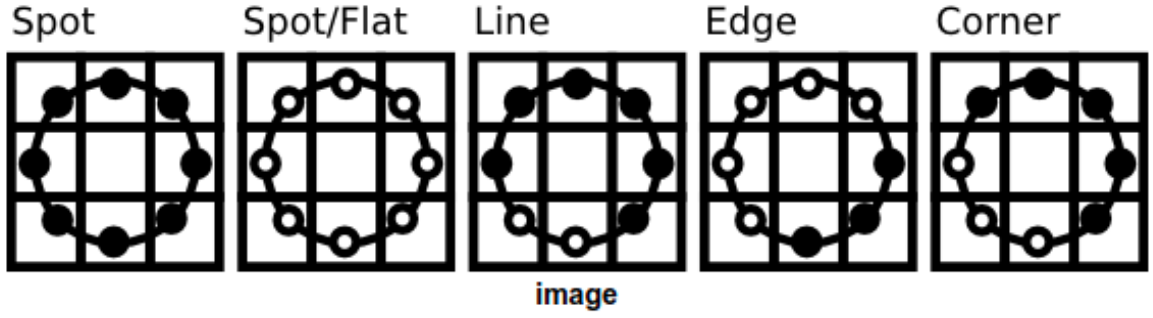


FIGURE 4.4 – Alignement de pixels sur un cercle

Pour un point donné (x_c, y_c) la position du voisin (x_p, y_p) , $p \in P$ peut être calculée par :

$$x_p = x_c + R \cos\left(\frac{2\pi p}{P}\right) \quad (4.21)$$

$$y_p = y_c + R \sin\left(\frac{2\pi p}{P}\right) \quad (4.22)$$

où R représente le rayon du cercle et P le nombre de points pris comme échantillon.

L'opérateur est une extension des codes LBP, il est donc parfois appelé "Extended LBP" (ou LBP circulaire). Si certaines coordonnées des points sur le cercle ne correspondent pas aux coordonnées de l'image, le point obtenu est interpolé.

L'implémentation OpenCV par exemple fait une interpolation bilinéaire :

$$f(x) \approx [x - 1, x] \begin{bmatrix} f(0, 0) & f(0, 1) \\ f(1, 0) & f(1, 1) \end{bmatrix} \begin{bmatrix} 1 - y \\ y \end{bmatrix}$$

Par définition, l'opérateur LBP est robuste contre les transformations monolithiques sur les échelles de gris. On peut facilement vérifier cela en examinant une image LBP extraite d'une image modifiée artificiellement.

Il reste donc à déterminer comment intégrer l'information spatiale dans le modèle de reconnaissance faciale. La représentation proposée par Ahonen et. al [Ahonen, 2006] consiste à diviser l'image *LBP* en m régions locales et à en extraire un histogramme. Le vecteur caractéristique amélioré spatialement est alors obtenu en concaténant les histogrammes locaux (sans les fusionner). Ces histogrammes sont appelés histogrammes locaux de motifs binaires. Voici un exemple ⁴ :

4. https://www.researchgate.net/figure/Enhanced-Local-Binary-Pattern-descriptor_fig1_295105395

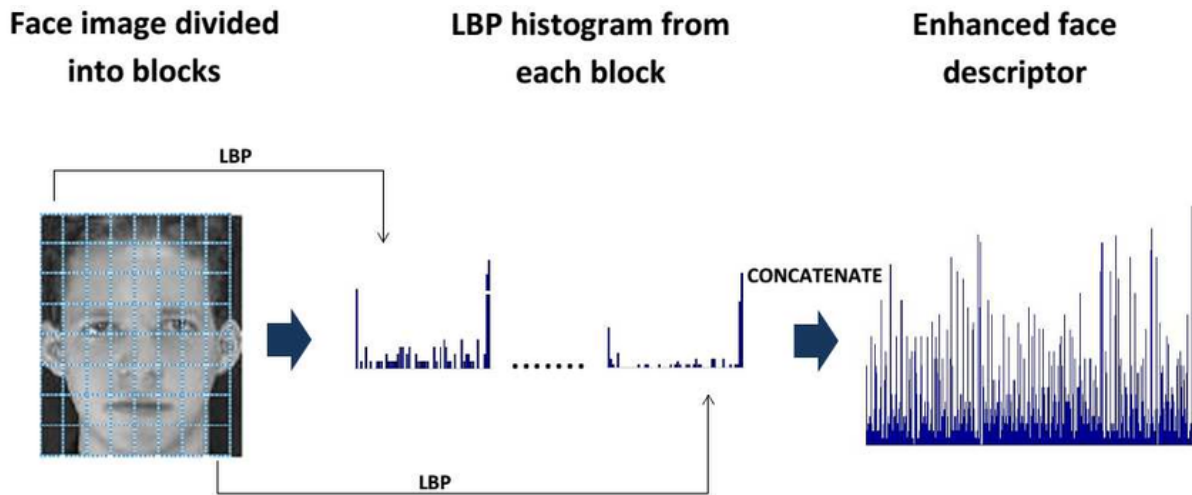


FIGURE 4.5 – Enhanced Local Binary histogram

4 CNN Face Recognition

Dans la deuxième partie du projet on va implémenter le module Keras (après amplification des données). Les données utilisées pendant la phase d'entraînement sont des images amplifiées à l'étape 1.2. [fig. 4.7](#)

A partir d'une seule image on obtient ainsi 10 différentes images sous des angles variables (zoom, rotations ...).

On teste ensuite la différence de performance entre les algorithmes de CNN (Convolutional Neural Network) et SVM (Support Vector Machine) pour les données utilisées.

Pour cela on cherche à implémenter les méthodes d'alignement facial, vecteurs d'intégrations, mesure du seuil de la distance, calcul de la précision, représentation schématique avec t-SNE ect...

4.1 Description du mode de fonctionnement de keras

Le modèle séquentiel est le modèle de base pour le réseau de neurones le plus simple dans Keras. Il nous permet de construire un modèle en empilant les couches de nœuds (neurones) les uns sur les autres.

Chaque argument du constructeur séquentiel est une couche de neurones, dans ce notre cas on n'a qu'une seule couche dense.

Considérant les couches denses (fortement connectées ou totalement connectées), tous les neurones reçoivent une entrée de tous les neurones présents dans la couche précédente. Cette couche dense ne comporte qu'un seul neurone (unités = 1) et l'entrée est une seule valeur (*input_shape*). Une représentation graphique du réseau de neurones séquentiel précédent est la suivante :

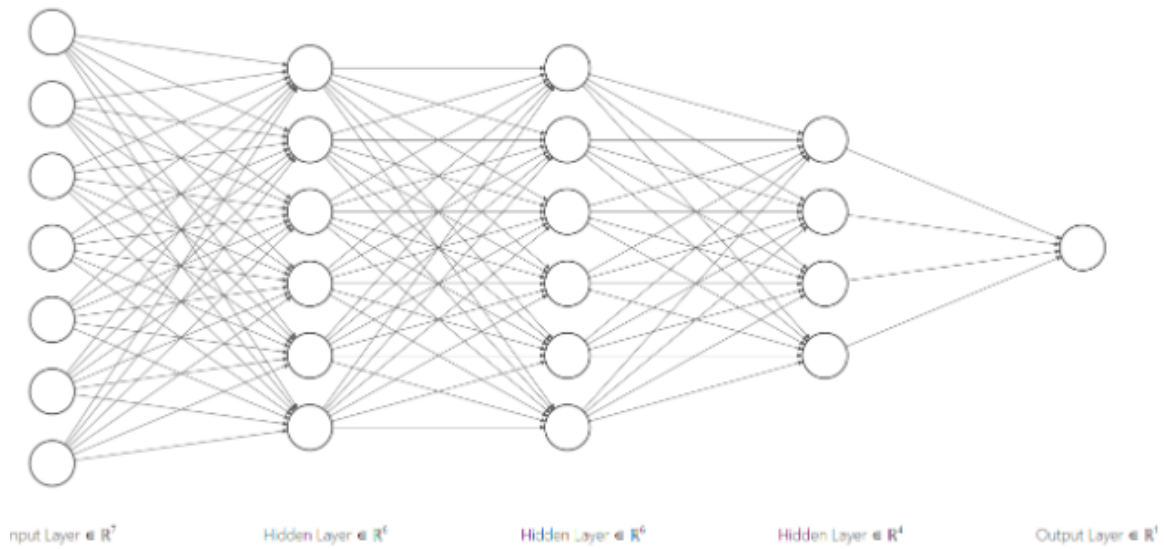


FIGURE 4.6 – Représentation graphique du réseau de neurone séquentiel

Chaque neurone a une fonction d'activation qui calcule la valeur transmise aux neurones de la couche suivante. Par défaut, les couches de Keras sont une fonction d'activation linéaire qui multiplie chaque entrée par le poids de chaque neurone et renvoie un signal de sortie proportionnel.

$$h(x) = \sum_{i=0}^n (\theta_i - x_i) \quad (4.23)$$

Dans ce cas on a une valeur à l'entrée x_1 et un poids aléatoire initial θ_1 mais on a aussi un biais avec une valeur initiale $x_0 = 1$ et un poids aléatoire initial θ_0 . Dans notre cas, la fonction d'activation du neurone unique est la fonction du modèle :

$$h(x) = \theta_0 + \theta_1 x_1 \quad (4.24)$$

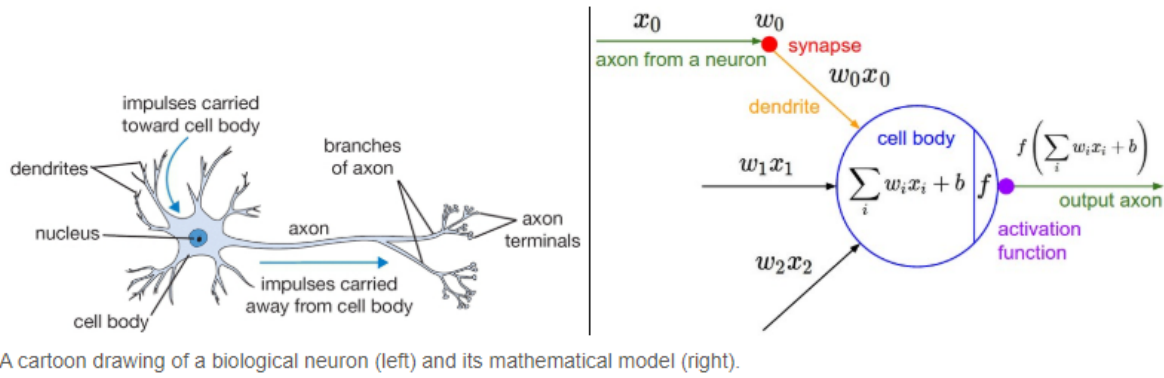


FIGURE 4.7 – Fonction d'activation du réseau neuronal

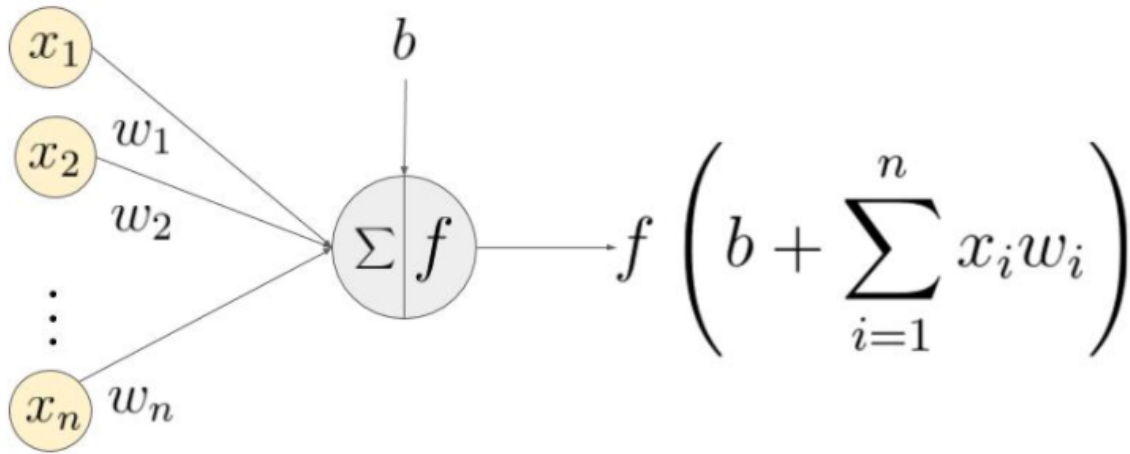


FIGURE 4.8 – Un exemple de neurone montrant l'entrée ($x_1 - x_n$) et les poids correspondants ($w_1 - w_n$), un biais (b) et la fonction d'activation f à la somme amortie des données à l'entrée.

Référence [fig. 4.7⁵](#) et [fig. 4.8⁶](#).

Un réseau de neurones a aussi besoin d'une fonction d'optimisation et une fonction de coût pour évaluer ses performances et ajuster les poids à chaque itération.

4.1.1 Loss Function

La fonction de coût d'un modèle renvoie une valeur qui représente la mesure des prédictions du modèle correspondent aux données d'entraînement. L'objectif est de minimiser la valeur retournée par la fonction de coût en ajustant les poids à chaque itération. Le modèle précédent utilise l'erreur quadratique moyenne (ESM) pour calculer cette valeur qui peut être modélisée

5. https://miro.medium.com/max/1400/1*BMSfaffFNEpqGFCNU4smPkg.png

6. <https://medium.com/swlh/activation-functions-in-artificial-neural-networks-8aa6a5ddf832>

par la formule suivante :

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (4.25)$$

Qui peut être aussi exprimée sous la forme :

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad (4.26)$$

Cela veut dire que si notre modèle prédit une valeur de 18000 alors que la valeur attendue est 20000, on a un MSE de 2000 (dans une seule itération).

4.1.2 Optimizer

La fonction d'optimisation représente la règle que les poids suivent pour ajuster leurs valeurs. Dans ce cas le modèle utilise *SGD* (Stochastic Gradient Descent). Cette fonction met à jour les poids itérativement de manière correspondante. Pour chaque poids θ_j :

$$\theta_j = \theta_j + \alpha \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (4.27)$$

α Représente le taux d'apprentissage, un paramètre qui reflète la manière avec laquelle les poids changent à chaque itération.

5 Analyse des données

La troisième étape du projet consiste à construire une base de données des entrées et sorties avec les images identifiées par l'algorithme de reconnaissance faciale. Pour chaque personne identifiée par la caméra on marque la date, l'heure d'entrée et de sortie ainsi que le nombre de passage par jour devant la caméra. Les données ainsi collectées sont analysées et on obtient une analyse statistique complète et représentative de la présence des étudiant en classe de façon journalière ou mensuelle par matière et par spécialité.

6 Visualisation des données avec PowerBI

On a choisi power BI comme outil pour l'analyse statistique des données car il est facile et rapide à implémenter sans besoin de connecter une base de donnée à un serveur en backend.

En plus il a l'avantage d'être interactif contrairement à d'autre plateformes d'analyses de données.

6.1 Représentation par spécialité

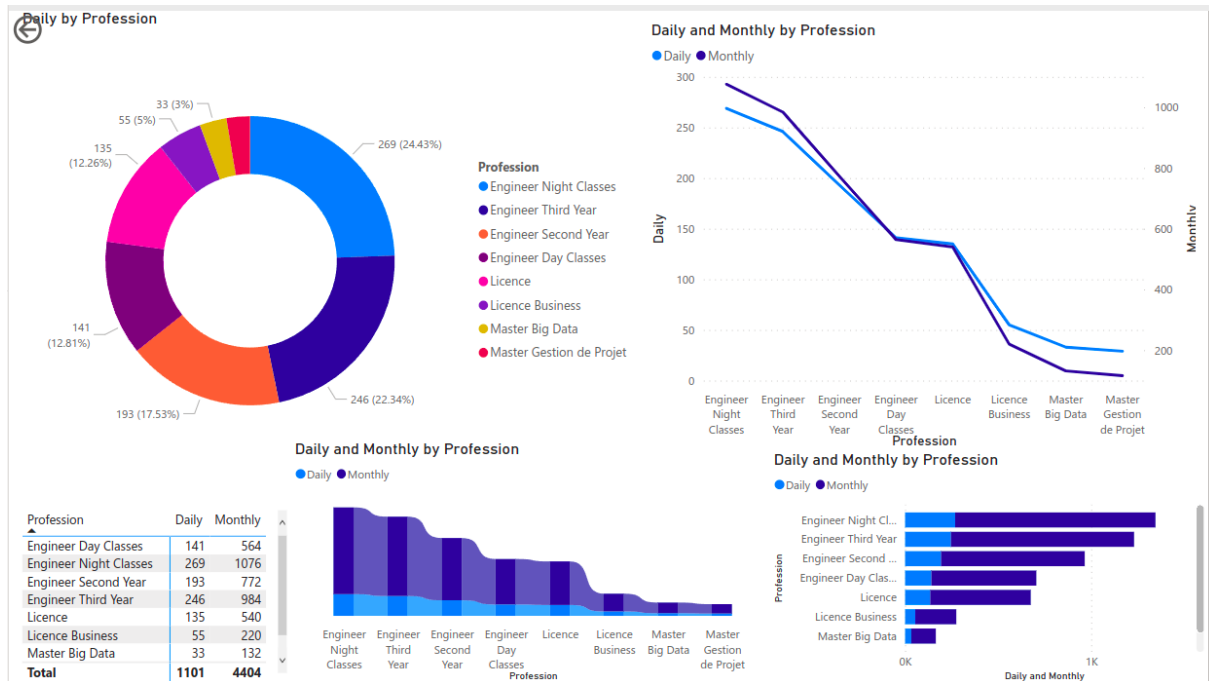


FIGURE 4.9 – Tableau de bord montrant le nombre de passage des étudiants par spécialités (les données utilisées ici sont des données théoriques juste à titre d'exemple)

Les données collectées par la caméras sont sous la forme d'un log journalier : Prenom, Nom, date, heure d'entrée et heure de sortie. Après passage, les résultats sont mis sous forme d'un tableau :

	Spécialité	Matière	Daily	Monthly	h.entrée	h.sortie
Prénom Nom 1	Ing 2	Sécurité	5	80	8 :30	16 :00
Prénom Nom 2	Ing 3	Cloud	4	70	10 :15	17 :20
Prénom Nom 3	Licence	Business	3	60	9 :15	15 :15

TABLE 4.1 – Tableau statistique des données collectés par l'algorithme de reconnaissance faciale

Chaque étudiant est classé par spécialité (année d'étude). La colonne matière représente la matière la plus fréquentée par l'étudiant le mois dernier. La colonne daily représente le nombre de passage par jour. La colonne monthly représente le nombre de passage par mois. h.entrée représente la première heure d'entrée (premier passage devant la caméra) et h.sortie représente la dernière heure de sortie.

On peut ainsi obtenir une statistique complète de chaque étudiant par spécialité et par matière.

6.2 Représentation par matière fréquentée



FIGURE 4.10 – Représentation statistique de la fréquentation journalière et par spécialité.

Une des interprétations qu'on peut tirer des histogrammes (les données étant à titre d'exemple) est qu'on observe une plus faible fréquentation des étudiants pour des matières comme l'anglais et les techniques de communications toutes spécialités confondues. Ce genre d'analyse nous permet d'avoir une vision claire pour chaque matière et spécialité afin d'adapter le volume horaire pour chaque étudiant selon son niveau et ses besoins.

7 En Résumé

L'application développée dans ce projet avait pour but principal de reconnaître les étudiants à l'entrée afin de pouvoir marquer la présence par matière de façon journalière et de comparer les résultats avec la feuille de présence marquée en classe. Pour cela nous avons besoin d'un algorithme de reconnaissance efficace qui pouvait faire preuve de bonne performance sous différentes conditions : angle de vue, inclinaison, luminosité, vitesse de passage, visibilité, distance etc ...

Les données devaient être accessibles rapidement (sous forme .xls) qu'on peut analyser librement sur power BI avec beaucoup plus de flexibilité et de personnalisation contrairement à d'autres plateformes en backend.

Le choix des bibliothèques de deep learning est important car les performances de l'algorithme va-

rien beaucoup avec la quantité de donnée. On a observé dans notre cas que pour un échantillon de 200 personnes enregistrés dans la base, on a une meilleure identification avec SVM qu'avec KNN avec une vitesse de calcul de l'algorithme presque identique.

La reconnaissance faciale en direct (live caméra) a l'avantage de dresser le profil d'une personne instantanément et peut être utilisée dans le cas où l'on a une visibilité directe et à courte distance (comme décrit à l'étape 1 du développement de l'application [fig. 4.7](#)).

Conclusion et perspectives

1 Conclusion d'étude

Le but des algorithmes de reconnaissance faciale consiste à dériver un ensemble de caractéristiques à partir des images qui minimise des variations intra-classe (se trouvant dans les instances d'un même individu) et maximise des variations extra-classe (images d'individus différents) [Sadedghi, 2010 ; Viola, 2001a]. Si les méthodes de calculs pour classer les images sont mal choisies, la performance du classifieur est considérablement impactée. Par conséquent, il est important de décider avec soin de la méthode à adopter lors de la conception de systèmes de reconnaissance d'image.

Idéalement, la représentation des traits devrait : (i) bien discriminer les images tout en tolérant les variations intra-classe ; (ii) être facilement extraites afin de permettre un traitement rapide ; et (iii) se situer dans un espace de faible dimension (courte distance vectorielle) afin d'éviter des calculs coûteux.

Naturellement, il n'est pas facile de travailler dans des conditions idéales où le classificateur peut performer avec une précision maximale de 100%. En effet, il est toujours important de tenir compte de la variabilité des différents facteurs d'imagerie lors de la capture des images tels que l'échelle, l'orientation, la pose, les expressions faciales, les conditions d'éclairage, la distance, etc... Il s'agit aussi d'autres critères aussi importants tels que : la détection, la localisation, l'enregistrement, et la vérification.

De nombreuses méthodes ont été proposées pour palier à ces problèmes. Les premières tentatives, comme les travaux de Kanade au début des années 70 [Patel, 2009], sont basées sur la représentation des visages en termes de caractéristiques géométriques, telles que les distances et les angles entre les repères etc ... Plus tard, des techniques basées sur l'apparence ont été proposées : Ces méthodes représentent généralement le visage comme un tableau 2D de pixels et visent à dériver des caractéristiques d'apparence sans avoir nécessairement recours à des calculs géométriques.

De même, différentes méthodes holistiques telles que l'analyse en composantes principales (ACP) [Turk, 1991], l'analyse discriminante linéaire (LDA) [Lades, 1993] et l'ACP 2D plus récemment [Schwarzer, 2000] ont été mises en place.

Les méthodes se basant sur les descripteurs locaux ont suscité une attention croissante en raison de leurs robustesse vis à vis des changements de pose et d'éclairage. Parmi ces descripteurs figurent les filtres de Gabor et les modèles binaires locaux [Starovoitov, 2002] qui se sont avérés très efficaces sur le terrain.

2 Ouverture et perspectives

Avec l'approfondissement de la recherche sur les réseaux neuronaux et l'apprentissage machine, la reconnaissance faciale est devenue une technique très utilisée dans différents aspects du quotidien. Les algorithmes sont assez performant à petite échelle et on peut atteindre des niveaux de précisions de 99% dans la plupart des cas. Par contre avec l'avènement de l'ère des méga données, les méthodes de reconnaissance faciale à petite échelle ne sont plus en mesure de performer de manière efficace et c'est pour cette raison que l'intérêt de développer des algorithmes de reconnaissance d'image permettant l'intégration de données volumineuses en temps réel est devenu le nouveau point central de la recherche en intelligence artificielle.

Documents en Annexes

Articles à comité de lecture

Projet Deep Learning PFE v1.2. *"Portraits de Reconnaissance Faciale Automatisée : Méthodes Deep Learning pour l'Analyse des Traits"*. iTeam University.

Extension Projet Cloud PFA v2.1 :. *"Connection du Framework Superset à Confluent (Kafka et Odoo) via Docker Bridges"*. iTeam University.

Application Map

Schéma détaillant les grandes parties de l'application mise en place dans ce projet.

Gantt Chart

Gantt chart suivi pendant le développement de l'application.

Power BI graphs

Exemple de tableaux de bords Power BI obtenu après analyse des données.

Bibliographie

- [Ahonen, 2006] T. AHONEN, A. HADID et M. PIETIKAINEN. « Face Description with Local Binary Patterns : Application to Face Recognition ». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 28.12 (2006), p. 2037-2041 (cf. p. 7, 38, 39).
- [Belhumeur, 1997] Peter N BELHUMEUR, Joao P HESPANHA et David J KRIEGMAN. « Eigenfaces vs. Fisherfaces : Recognition Using Class Specific Linear Projection ». *IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE* 19.7 (1997), p. 10 (cf. p. 4, 34, 35).
- [Bichsel, 1994] M. BICHSEL et A. P. PENTLAND. « Human face recognition and the face image set's topology ». *CVGIP : Image Understanding* 59.2 (1994), p. 254-261 (cf. p. 11).
- [Brunelli, 1993] Roberto BRUNELLI et Tomaso POGGIO. « Face Recognition : Features Versus Templates ». *Pattern Analysis and Machine Intelligence, IEEE Transactions on* 15 (1993), p. 1042-1052 (cf. p. 4).
- [Chen, 2013] Chi Hau CHEN. « An introduction to computer vision in medical imaging ». *Computer Vision in Medical Imaging*. T. Volume 2. Series in Computer Vision Volume 2. WORLD SCIENTIFIC, 2013, p. 1-16 (cf. p. 2).
- [Cootes, 2000] T.F. COOTES, K. WALKER et C.J. TAYLOR. « View-based active appearance models ». *Proceedings Fourth IEEE International Conference on Automatic Face and Gesture Recognition (Cat. No. PR00580)*. Proceedings Fourth IEEE International Conference on Automatic Face and Gesture Recognition (Cat. No. PR00580). 2000, p. 227-232 (cf. p. 5).
- [Guo, 2000] G. GUO, S. LI et K. CHAN. « Face recognition by support vector machines ». *Proceedings Fourth IEEE International Conference on Automatic Face and Gesture Recognition (Cat. No. PR00580)* (2000) (cf. p. 5).
- [Heitmeyer, 2000] Roderick HEITMEYER. « Biometric identification promises fast and secure processing of airline passengers ». *ICAO journal*. (2000), p. 11 (cf. p. 9).
- [Lades, 1993] M. LADES, J.C. VORBRUGGEN, J. BUHMANN, J. LANGE, C. von der MALSBERG, R.P. WURTZ et al. « Distortion invariant object recognition in the dynamic link architecture ». *IEEE Transactions on Computers* 42.3 (1993), p. 300-311 (cf. p. 47).
- [Lawrence, 1997] S. LAWRENCE, C.L. GILES, Ah Chung TSOI et A.D. BACK. « Face recognition : a convolutional neural-network approach ». *IEEE Transactions on Neural Networks* 8.1 (1997), p. 98-113 (cf. p. 6).
- [Lienhart, 2002] R. LIENHART et Jochen MAYDT. « An extended set of Haar-like features for rapid object detection ». *Proceedings. International Conference on Image Processing* (2002) (cf. p. 4).
- [Moghaddam, 2000] Baback MOGHADDAM, Tony JEBARA et Alex PENTLAND. « Bayesian face recognition ». *Pattern Recognition* 33.11 (2000), p. 1771-1782 (cf. p. 6).
- [Murase, 1995] Hiroshi MURASE et Shree K. NAYAR. « Visual learning and recognition of 3-d objects from appearance ». *International Journal of Computer Vision* 14.1 (1995), p. 5-24 (cf. p. 13).
- [Nefian, 1996] Ara V. NEFIAN. *Statistical Approaches To Face Recognition*. 1996 (cf. p. 4).
- [O'Toole, 2002] Alice J. O'TOOLE, Dana A. ROARK et Hervé ABDI. « Recognizing moving faces : a psychological and neural synthesis ». *Trends in Cognitive Sciences* 6.6 (2002), p. 261-266 (cf. p. 14).
- [Patel, 2009] Shwetak N. PATEL, Jay W. SUMMET et Khai N. TRUONG. « BlindSpot : Creating Capture-Resistant Spaces ». *Protecting Privacy in Video Surveillance*. Sous la dir. d'Andrew SENIOR. London : Springer, 2009, p. 185-201 (cf. p. 47).
- [Penev, 1996] Penio S PENEV et Joseph J ATICK. « Local feature analysis : a general statistical theory for object representation ». *Network : Computation in Neural Systems* 7.3 (1996), p. 477-500 (cf. p. 7).

- [Raudys, 1991] Sarunas RAUDYS. « Small Sample Size Effects in Statistical Pattern Recognition : Recommendations for Practitioners ». *Pattern Analysis and Machine Intelligence, IEEE Transactions on* 13 (1991), p. 252-264 (cf. p. 35).
- [Roark, 2006] Dana A. ROARK, Alice J. O'TOOLE, Hervé ABDI et Susan E. BARRETT. « Learning the moves : the effect of familiarity and facial motion on person recognition across large changes in viewing format ». *Perception* 35.6 (2006), p. 761-773 (cf. p. 15).
- [Sadeghi, 2010] Ahmad-Reza SADEGHI, Thomas SCHNEIDER et Immo WEHRENBURG. « Efficient Privacy-Preserving Face Recognition ». *Information, Security and Cryptology – ICISC 2009*. Sous la dir. de Donghoon LEE et Seokhie HONG. Lecture Notes in Computer Science. Berlin, Heidelberg : Springer, 2010, p. 229-244 (cf. p. 47).
- [Schroff, 2015] Florian SCHROFF, Dmitry KALENICHENKO et James PHILBIN. « FaceNet : A Unified Embedding for Face Recognition and Clustering ». *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, p. 815-823. arXiv : [1503.03832\[cs\]](#) (cf. p. 17, 19, 20).
- [Schwarzer, 2000] Gudrun SCHWARZER. « Development of Face Processing : The Effect of Face Inversion ». *Child Development* 71.2 (2000), p. 391-401 (cf. p. 47).
- [Starovoitov, 2002] Valery STAROVOITOV et D BRYLIUK. « Access control by face recognition using neural networks ». 2002 (cf. p. 48).
- [Szegedy, 2014] Christian SZEGEDY, Wei LIU, Yangqing JIA, Pierre SERMANET, Scott REED, Dragomir ANGUELOV et al. *Going Deeper with Convolutions*. arXiv :1409.4842. arXiv, 2014. arXiv : [1409.4842\[cs\]](#) (cf. p. 19).
- [Turk, 1991] M.A. TURK et A.P. PENTLAND. « Face recognition using eigenfaces ». *1991 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Proceedings*. 1991 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Proceedings. 1991, p. 586-591 (cf. p. 47).
- [Turk, 2001] Matthew TURK. *A Random Walk Through Eigenspace*. 2001 (cf. p. 4, 11).
- [Viola, 2001a] Paul VIOLA et Michael JONES. « Robust Real-Time Object Detection ». *International Journal of Computer Vision - IJCV*. T. 57. 2001 (cf. p. 47).
- [Viola, 2001b] Paul A. VIOLA et Michael J. JONES. « Rapid object detection using a boosted cascade of simple features ». *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001* (2001) (cf. p. 4).
- [Yambor, 2000] Wendy S. YAMBOR, Wendy S. YAMBOR et Wendy S. YAMBOR. *Analysis of pca-based and fisher discriminant-based image recognition algorithms*. 2000 (cf. p. 10).
- [Zhao, 2003] W. ZHAO, R. CHELLAPPA, P. J. PHILLIPS et A. ROSENFELD. « Face recognition : A literature survey ». *ACM Computing Surveys* 35.4 (2003), p. 399-458 (cf. p. 5).

RÉSUMÉ

Un système de reconnaissance faciale est une technologie capable de faire la correspondance entre les visages présent sur une image numérique avec une base de données préalable. Généralement utilisée pour s'assurer de l'identité des personnes dans les processus d'authentification et de sécurité biométriques, ces algorithmes fonctionnent en identifiant et en mesurant les traits des visages à partir d'une image donnée.

Les recherches sur les algorithmes de la reconnaissance faciale ont commencé dans les années 1960 mais ce n'est que récemment qu'on a vu un essor considérable dans le déploiement des systèmes de reconnaissance faciale.

Non seulement les technologies de vision par ordinateur sont plus faciles à mettre en place de nos jours mais elles nous permettent également de mieux discerner des images dans des conditions extrêmes ou qui posent défi.

Étant donné que la reconnaissance faciale informatisée implique la mesure des caractéristiques physiologiques d'un être humain, les systèmes de reconnaissance faciale sont classés comme biométriques. Bien que la précision des systèmes de reconnaissance faciale en tant que technologie biométrique soit inférieure à la reconnaissance de l'iris et aux empreintes digitales, elle est largement adoptée en raison de son mécanisme qui permet d'éviter le contact humain.

MOTS CLÉS

image detection, recognition, classification, optimisation

ABSTRACT

Deux des aspects importants dans le cadre de recherche sur la reconnaissance faciale par ordinateur sont abordés ici : la détection des visages et des caractéristiques faciales d'une part, et les méthodes de comparaison et d'identification des visages d'autre part. Les meilleurs résultats apportés au problème de reconnaissance faciale sont obtenus avec les méthodes deep learning notamment avec les classifieurs SVM. Dans cette approche, la détection globale des visages, la localisation des caractéristiques faciales et la comparaison des visages sont effectuées en une seule étape. Ce rapport décrit la mise en place d'une application de détection faciale et d'analyse de données. D'une part, nous décrivons une technique d'apprentissage visuel et la manière avec laquelle on peut l'appliquer à la détection de visage dans diverses conditions. D'autre part, un algorithme de tri et d'analyse des données est implémenté afin de présenter des résultats en temps réel et permettre leur analyse statistique. Enfin, nous apportons une extension au projet en connectant la base de données à la plateforme Cloud via les connecteurs GDBC de Confluent^a qui permettent d'automatiser l'analyse des données en temps réel (voir article en annexe^b).

a. <https://www.confluent.io/fr-fr/product/confluent-connectors/>

b. Projet Cloud PFA v2.1 : Connection du Framework Superset à Confluent via Docker Bridges

KEYWORDS

computer vision, perception, modelisation